

Aristotle University of Thessaloniki
School of informatics

Distributed computation engine

Theodoros Kaimasidis

Supervisor: Petros Nicopolitidis

September 30, 2026

Abstract

This work is trying to solve a fundamental issue, that of combining many computing systems in order to create one of larger capacity, without however creating something that is embarrassingly-parallel. The exposure to the underlying mechanisms of the systems involved is also a welcome challenge, and with this I want to distill experience that would be hard to amass in a professional setting with the exciting field of research in the area of massive computation. Inspiration is drawn from the distributed systems applicable to commercial applications and software, as well as the more scientific endpoints of linux and the fundamental technologies used. The result resembles a RPC framework, with reusable and extensible components that for the most part bond well to the underlying structure of the orchestrator that manages the lifecycles of the userspace programs we are going to be using to have run the set of computations, of which results we are going to base ourselves on for the evaluation.

Contents

Abstract	i
1 Introduction	1
1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks?	1
1.2 The hard part	1
1.3 Structure of this Thesis	2
2 Systems Programming Foundations: C and Inter-Process Communication	3
2.1 C: the abstract machine and the object model	3
2.2 Pointers, arrays and layout	4
2.3 Functions, function pointers and the ABI	4
2.4 Memory, ownership and lifetime	4
2.5 Processes, descriptors and signals	5
2.6 Inter-process communication: the families	5
2.7 System V IPC	5
2.8 POSIX IPC	6
2.9 Comparing the mechanisms	7
2.10 Controlling many processes from one	8
2.11 Dynamically connected client-server pairs	9
3 Containers, Orchestration, and Scaling	11
3.1 Docker and the container model	11
3.2 Kubernetes: orchestration	14
3.2.1 kubectl / kubeapiserver	15
3.2.2 etcd	15
3.2.3 kube-scheduler	16
3.2.4 Cluster Autoscaler / Karpenter	16
3.2.5 Worker Node	17
3.2.6 kubelet	17
3.2.7 Container Runtime	17
3.2.8 kube-proxy	17
3.2.9 How the Kinds Relate	18
3.2.10 Pod	18
3.2.11 ReplicaSet	19
3.2.12 Deployment	19
3.2.13 StatefulSet	20
3.2.14 DaemonSet	21
3.2.15 Job	22
3.2.16 CronJob	22
3.2.17 Service	23

3.2.18	ConfigMap & Secret	23
3.3	The object model	24
3.4	API gateways, ingress, and service mesh	25
3.5	The engine on Kubernetes: which entities it uses, and for what	25
3.5.1	Pod: one pod is one engine node	26
3.5.2	StatefulSet rather than Deployment	26
3.5.3	Service: headless, for naming only	27
3.5.4	Job: a batch, submitted and gone	27
3.5.5	ConfigMap and the downward API	27
3.5.6	Autoscaling: KEDA, the HPA and metrics-server	28
3.5.7	Scheduler, kubelet and runtime, and the network	28
3.5.8	What is deliberately not used	28
3.5.9	An alternative split: one head pod, many worker pods	29
4	Two Transports: XDP and gRPC	31
4.1	XDP: programmable packet processing at the driver	32
4.1.1	eBPF, maps, and the verifier	32
4.1.2	Attachment modes and the reality of zero copy	33
4.1.3	AF_XDP: the UMEM and its four rings	33
4.1.4	How this project uses XDP	33
4.1.5	Frame ownership across the pool, and in-kernel admission	34
4.1.6	Deployment: privileges and containers	34
4.2	gRPC: typed RPC on HTTP/2	34
4.3	XDP, io_uring and shared-memory IPC, together	35
4.4	Where each fits	36
5	Protocol Buffers	38
5.1	The schema and code generation	38
5.2	The wire format	38
5.3	Schema evolution	39
5.4	Implementing the wire format	39
5.4.1	Forward compatibility is one line	39
5.4.2	Default elision, and what it costs	40
5.4.3	The fiddly parts	40
5.4.4	Checking the claim	41
5.5	gRPC and Protocol Buffers together	41
6	Coordination: Algorithms and the Services That Implement Them	42
6.1	Why agreement is hard	42
6.2	Consensus: Paxos, and then Raft	42
6.3	Leases, heartbeats, and membership	43
6.4	The services	43
6.5	Gossip, when agreement is not required	43
6.6	Brokers: Kafka and RabbitMQ	44
6.7	Placement without a directory	44
6.8	What this project uses, and why	44
7	The Engine	46
7.1	The high level design	46
7.2	The job model	47
7.3	One wire format	48
7.4	Identity: ids computed from the bytes	48

7.5	Placement by reverse traversal	49
7.6	Dependency tracking without consensus	50
7.7	Shipping code	50
7.8	Two heads: C and Java	50
7.9	Deployment	51
7.10	What the engine demonstrates, and what it does not	51
8	The Distributed Engine: Architecture and Alternatives	53
8.1	What changed, and why	53
8.2	The object model	53
8.3	Identity	55
8.4	The node	55
8.4.1	The worker	55
8.4.2	The filesystem as the reply path	56
8.4.3	The head	57
8.5	The cluster layer	58
8.5.1	What is stored	58
8.5.2	Leases, or how a node leaves	58
8.5.3	Publishing the difference	58
8.5.4	Moving work	58
8.6	What the two-node experiment establishes	59
8.7	Technologies considered and not used	60
8.7.1	Raft, implemented rather than consumed	60
8.7.2	ZooKeeper and Consul	61
8.7.3	Gossip instead of consensus	61
8.7.4	A distributed hash table	61
8.7.5	A message broker	61
8.7.6	gRPC, and other serialisation formats	61
8.7.7	MPI and the HPC tradition	62
8.7.8	Existing distributed frameworks	62
8.7.9	Kubernetes as the scheduler	62
8.7.10	A shared filesystem or object store	62
8.7.11	Faster transports	62
8.8	Limitations and what attaches next	63
9	Scaling the Parts Apart, and a Controller of Our Own	65
9.1	Heads and workers as separate pods	65
9.1.1	Why the two parts want to scale differently	65
9.1.2	What the engine needed	66
9.1.3	How it is deployed	66
9.1.4	What it changes, and what was shown	66
9.1.5	What it costs	67
9.2	The case for a controller of our own	67
9.2.1	What the orchestrator costs this engine	67
9.2.2	The same responsibilities, from the operating system	68
9.2.3	Kubernetes, and what replaces it on one machine	69
9.2.4	The case, and its limits	69
9.3	Bridging the two levels: KEDA and CRIU	69
9.4	Notions from distributed systems worth implementing	70

Chapter 1

Introduction

This thesis is about computation, systems, distributed computation and their intertwining in an interesting project. The need is accelerated computation via the means of one or more computers/computation nodes, which may have lost some of its importance in our days, thanks to many available multi-core consumer and server computer processors, but it's one aspect of computing systems that's extremely rewarding and has too much to teach in the process of its implementation. The problem visited is a personal question of mine from when I was way younger, how can I combine two slow machines to run something faster.

1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks?

This is something that many pc enthusiasts dream of doing, combining the power of multiple cpus or computers for more compute, and it's actually been a huge trend with the advent of data centers and distributed systems evolution and adoption on the business of software development and deployment. The case reviewed here is one about normal pcs with clear boundaries and store bought interconnects (Ethernet) facilitating their intercommunication. The first challenge met is the design of the system, unlike the 'system-design' paradigm of today's development circles however the architecture here can scale up and down, traverse and live in different layers. For example computation is discussed in the context of an os, also in the process context of the C runtime and also with the linux operating system being open source we have access to both with one tool: the C programming language. Further, concurrency is a "solved" issue, the problem is the communication and mutual exclusion of what constitutes a thread in our application, it's been a struggle designing something that is not a mere "embarrassing-parallel" machine, while it's been a similar note for the application of the distributed systems side of algorithms for consensus and communication between nodes.

1.2 The hard part

The hard part comes before the implementation, the decision of what to focus on, what to work on, what to reuse and what to write ourselves. The result of this work is software that can run on two or more machines and via a gateway obfuscate the raw computing power of both to provide higher compute than each of the constituents, hopefully in a scalable way (and in a scalable way that makes sense). For a start we need to define the "computation", what exactly is going to be worked up on: the choice has been made for this to be a set of input data and a function that's to run on them. This comes instead of a process, which in linux is completely open to a developer to see, even if there needs to be a kernel loadable module. Furthermore, this pair constitutes a "job", and the objective is to be able to distribute them in the environments that we control. A

plain worker written in C or Java for example would not suffice here, it's clear that there is a need of a head and a worker component on each installation. As a result after the choice of the execution mechanism, what is needed is a routing algorithm, or a choice of a routing strategy in order to be able to provide an actual higher capacity for computations across two systems, which in our finalized project is taken care of the head (written in C) that constitutes a distributed system. And finally, the landing ground for these has been set to be the kubernetes orchestrator engine, which means packaging in containers and managing of the compute nodes accordingly, this choice has been made because the philosophy of this technology perfectly contributes to the idea of the pooling together of the resources of multiple computers and also is in and of itself an interesting endeavor since much of the modern infrastructure operates in the form of a kubernetes cluster. What has been clear from the start of the project however is the same as what struck me most when I first studied concurrency, running threads in parallel is easy, initiating multiple processes is also easy, problems arise when we have exclusivity and in this way bottlenecks.

1.3 Structure of this Thesis

This thesis puts into context the software project that is the showcase of the work. In total it explains the technologies used, explores the underlying concepts and the mechanisms that are exposed via apis or the ones that are applicable to the project in a meaningful way. A more general note of exploration and care has been given because the whole project is a reflection of my own personal interests.

In so doing, the first chapters covers the mechanism that linux has that handle execution and process creation and management, what is a process what it sees and what it's universe comprises of is covered, and along with the syscalls and c sections the foundations of the project are set. **Chapter 2 is that foundation: the C language as the engine relies on it — the abstract machine, the object model, pointers, layout, the function-pointer ABI and memory ownership — followed by processes and signals, System V and POSIX inter-process communication with their kernel data structures, the controller patterns for managing many processes from one, and the way operating systems pair clients with servers dynamically.**

Later on we discuss the communication framework that's to be used in the Protocol Buffer and RPC chapter, along with the overall effect on the performance of the system, how they work and how they can be applied.

In a separate chapter XDP is covered, a special project that's part of the eBPF family of programs that's seen recent popularity and applicability on a huge number of projects both commercial and research focused.

Docker and Kubernetes have their own dedicated chapters, along with mentions on collaborating projects CRIU and Karpenter that are used in the project, facilitating process migration and "compute node" discovery.

Finally, we have a theoretical chapter on distributed systems and their theory, major algorithms are being mentioned and their usages on our project explored, accompanied in the end with the implementation chapter: whose sole purpose is to give a high level explanation of the architectural decisions made and the routes taken toward the end goal. **The last chapter, Chapter 9, takes two further steps: it splits the node so that heads and workers scale as separate pods, and then makes the case for replacing the orchestrator, on each machine, with a controller of the engine's own built from the inter-process communication of Chapter 2.**

Chapter 2

Systems Programming Foundations: C and Inter-Process Communication

Everything later in this thesis rests on two layers that are easy to take for granted: the C language, in which the worker, the wire format and every job function are written, and the operating system’s mechanisms for letting processes talk to one another. This chapter sets both down. The C part follows the author’s own notes on the language’s mechanics [2]; the systems part follows Kerrisk’s treatment of the Linux programming interface [1]. Where a rule is stated, the place in the engine that depends on it is named, so that the later chapters can refer back rather than re-explain.

2.1 C: the abstract machine and the object model

A C program does not describe what the processor does; it describes the behaviour of an *abstract machine*, and the compiler is free to emit any code whose observable behaviour matches that machine’s — for programs whose behaviour is defined. Undefined behaviour is therefore not “whatever the hardware does”: it is a promise to the abstract machine that the program broke, after which the optimiser owes it nothing. When correct-looking code is “broken by the compiler”, the first question is which promise was violated.

Objects, storage duration and linkage. Every object has a storage duration — static (the whole run), automatic (a block), allocated (`malloc` to `free`) or thread — and every name a linkage. Headers carry `extern` declarations; definitions live in exactly one `.c` file; and a function defined in a header is written `static inline`, so that every translation unit may include it without producing duplicate symbols. The engine’s `contract.h` is built on exactly this rule: the worker, the head and every job function compile against the same header of `static inline` accessors, which is what makes a second, drifting copy of the format impossible on the C side.

Effective type and aliasing. An object may be read only through an lvalue of its own (effective) type, a signed or unsigned variant of it, an aggregate containing it, or a character type. Reading a `uint32_t` out of a received byte buffer by casting the buffer’s pointer violates that rule, and on some processors the address is misaligned as well. The portable alternative is to copy the bytes:

```
static inline uint32_t readUint32At(const void *object, uint32_t offset) {
    uint32_t value; memcpy(&value, (const unsigned char *)object + offset, 4);
    return value;
}
```

The `memcpy` costs nothing — compilers turn a fixed four-byte copy into a single load — and it is defined for any alignment. It also fixes the *object representation* as the one the machine

uses: on x86-64 that is little-endian, which is why the engine’s wire format is little-endian and why a Java head must reorder every buffer it reads (Section 7.8).

Integer arithmetic. Unsigned arithmetic wraps modulo 2^n ; signed overflow is undefined. A size computed as `rows * stride` from values read off the wire can therefore wrap to a small number, pass a bounds check, and cause an allocation far smaller than the loop that fills it. The only safe order is to bound the factors *before* multiplying — the rule the engine’s `dsBytes` follows, and the bug the earlier prototypes shared (Section 7.3).

2.2 Pointers, arrays and layout

Pointer arithmetic is defined only inside one array object, and only for results from its first element to one past its last; the one-past-the-end pointer may be formed and compared but never dereferenced, and a single object counts as an array of one. Arrays decay to pointers to their first element when passed, so `int **` and `int [4] [6]` are different, incompatible types. `void *` can carry any *object* pointer and give it back unchanged, but the standard does not promise the same for function pointers — a gap that POSIX closes for `dlsym`, and that the engine sidesteps by copying the returned address into a function-pointer variable with `memcpy`.

A **struct** is laid out in declaration order, with padding inserted so that each member meets its alignment; declaring members in decreasing order of alignment wastes the least. Padding is also why the engine does not describe its objects as packed structures to be written straight to a socket: the layout would depend on the compiler, and an overlay cannot validate a field before it is used. The format is a list of offsets instead, each field read and checked on its own. A **union** is safe only beside a tag that says which member is live — exactly the role the `type` field of every object header plays for the four object kinds.

2.3 Functions, function pointers and the ABI

A declaration mimics its use, which makes function-pointer types unreadable unless named; every function-pointer type used more than once deserves a `typedef`. The engine has exactly one that matters:

```
typedef int64_t (*JobFunction)(const void *payload, uint32_t payloadLength,
                               void *output, uint32_t outputCapacity);
```

It is the whole contract between the engine and shipped code: a function sees bytes in and a buffer out, returns the bytes written or a negative value on failure, and knows nothing of identity or placement. Because the code arrives as a shared object and is called through this pointer after `dlopen/dlsym`, both sides must agree on the platform’s calling convention — on x86-64 Linux the System V ABI, which passes these four arguments in `rdi`, `rsi`, `rdx` and `rcx` and returns in `rax`. That agreement is what `ENGINE_ABI_VERSION` names: a shared object built against a different signature is refused before it is ever called. Callback interfaces in general should also pass an opaque `void *ctx` through unchanged, or their users are pushed into global state; the worker’s thread pool does this by handing each task its own copied payload.

2.4 Memory, ownership and lifetime

C has no owner, so a program must decide one for every allocation and write the decision down. A few habits carry most of the weight: allocate as `malloc(n * sizeof *p)`, so the size cannot go stale when the type changes; mark every pointer member of a structure as owned or borrowed; release resources through a chain of cleanup labels named after what they undo; and set a pointer to `NULL` after freeing it, which turns a silent use-after-free into a deterministic crash.

The engine’s object store takes the other approach to lifetime: ownership by region. It maps one large area with `mmap(MAP_NORESERVE)` — address space only, backed by the kernel page by page as it is touched — and carves payloads and records out of it with a bump allocator. Nothing is freed individually, so no pointer the store hands out can dangle while the store exists, and destroying the store is a single `munmap`.

Undefined behaviour and memory errors are caught with tools as much as with care: `-Wall` `-Wextra` and more as a baseline, AddressSanitizer for out-of-bounds and use-after-free, UndefinedBehaviorSanitizer for overflow and misaligned access, Valgrind for reads of uninitialised memory, and ThreadSanitizer, run separately, for data races.

2.5 Processes, descriptors and signals

A process is created with `fork`, which duplicates the caller, and given a new program with `execve`; its parent collects its exit status with `waitpid`, and until it does, the exited child remains a zombie. Every open channel — file, pipe, socket, POSIX message queue — is a small integer, a *file descriptor*, indexing a per-process table that `fork` copies; this is how a parent hands an already open channel to its child.

Signals are asynchronous notifications. They are installed with `sigaction`, and a handler may do almost nothing safely: set a volatile `sig_atomic_t` flag, or call one of the few async-signal-safe functions. Whether an interrupted system call is resumed is itself a choice: with `SA_RESTART` a blocking `accept` simply continues, without it the call returns `EINTR`. The engine’s worker installs its `SIGINT/SIGTERM` handler *without* `SA_RESTART` for precisely this reason — so that a stop request wakes the accept loop instead of being absorbed by it. A controller that starts workers with `fork` handles `SIGCHLD` and reaps with `waitpid(-1, &status, WNOHANG)` in a loop, since several children may exit behind one signal.

2.6 Inter-process communication: the families

Processes share nothing by default, so every exchange between them goes through a kernel mechanism. Linux offers several, which differ along two axes: whether they move *data* or only *synchronise*, and whether they are reached by a file descriptor — and can therefore be waited on together with other descriptors in `poll` or `epoll` — or by an identifier of their own.

mechanism	reached by	nature
pipe, FIFO	descriptor	byte stream, one direction; a FIFO has a name in the filesystem
UNIX domain socket	descriptor	stream or datagram, both directions, local; can pass descriptors
System V message queue	identifier	discrete typed messages, kernel persistence
System V semaphore set	identifier	counters with atomic multi-operation updates
System V shared memory	identifier	a memory segment attached into several address spaces
POSIX message queue	descriptor	discrete messages with priorities; pollable on Linux
POSIX semaphore	name or memory	a counter; futex-based, cheap when uncontended
POSIX shared memory	descriptor	a tmpfs object mapped with <code>mmap</code>
signal	process id	a notification carrying almost no data

2.7 System V IPC

The three System V mechanisms share one shape. An object is found by a `key_t`, usually derived from a path and a byte with `ftok`, or is created unnamed with `IPC_PRIVATE`; a `...get` call turns the key into an integer identifier, creating the object if `IPC_CREAT` is given and failing if it already

exists when `IPC_EXCL` is added; a `...ctl` call inspects it, changes its permissions, or removes it with `IPC_RMID`. The objects have *kernel persistence*: they outlive the processes that created them until removed explicitly or until reboot, and `ipcs` and `ipcrm` list and remove leftovers. Identifiers are not file descriptors, so a process cannot wait on a message queue and a socket in the same `poll`.

Message queues (`msgget`, `msgsnd`, `msgrcv`) carry discrete messages whose first field is a positive `long`, the message *type*:

```
struct request { long mtype; pid_t client; int command; };    /* mtype > 0 */

msgsnd(queueId, &req, sizeof req - sizeof req.mtype, 0);    /* size excludes mtype */
msgrcv(queueId, &reply, sizeof reply - sizeof reply.mtype, getpid(), 0);
```

`msgrcv` can take the first message of any type (`msgtyp 0`), of one exact type (`> 0`), or of the lowest type up to a bound (`< 0`). Using the client's process id as the type lets many clients share one reply queue, each reading only its own replies.

Semaphores (`semget`, `semop`, `semctl`) come in *sets*, and `semop` applies an array of operations to a set atomically — all or none — which makes acquiring several resources at once deadlock-free. `SEM_UNDO` asks the kernel to reverse a process's adjustments if it dies holding them. Creation and initialisation are separate calls, so a second process can find a set that exists but has not yet been given its initial values; the usual remedy is to have exactly one creator, chosen by `IPC_EXCL`, initialise it before announcing it.

Shared memory (`shmget`, `shmat`, `shmdt`) creates a segment that each process attaches into its own address space — possibly at a different address in each, so structures placed in it must hold offsets rather than pointers. `IPC_RMID` marks a segment for destruction; it disappears when the last process detaches.

Inside the kernel (Linux 4.4 source), every System V object begins with a `struct kern_ipc_perm` — its key, owner and creator ids, mode, a sequence number and a `deleted` flag — and each IPC namespace keeps, per mechanism, a `struct ipc_ids` that maps identifiers to objects through an IDR (an integer-keyed radix tree). The identifier a program sees is `index + sequence × SEQ_MULTIPLIER`, so a slot that is freed and reused yields a different identifier and a stale one fails rather than reaching the new object. The three objects extend the common header:

- `struct msg_queue`: byte and message counts, the queue's limit, the last sender and receiver, and three lists — the queued messages, the receivers blocked waiting for one, and the senders blocked on a full queue;
- `struct sem_array`: the array of semaphores, the number of them, lists of pending operations (those that alter values and those that only wait for them), and the undo records of `SEM_UNDO`;
- `struct shmid_kernel`: a `struct file` on the kernel's internal `shmem` filesystem that holds the pages, the segment size, the number of current attaches, and the creating task.

IPC namespaces (`CLONE_NEWIPC`) give each container its own copy of these tables, which is why System V objects of one container are invisible to another; the containers of one Kubernetes pod share an IPC namespace, so two processes in one pod can use them to talk to each other (Chapter 3).

2.8 POSIX IPC

The POSIX interfaces replace keys with names of the form `/name` and return, where they can, file descriptors.

Message queues (`mq_open`, `mq_send`, `mq_receive`, `mq_close`, `mq_unlink`) deliver the oldest message of the highest priority first, with limits set in a `struct mq_attr`; `mq_notify` can deliver a

signal or start a thread when a message arrives on an empty queue. On Linux a queue descriptor is a real file descriptor on the `mqueue` filesystem, so it can be watched with `epoll` alongside sockets — the practical advantage over System V queues.

Semaphores are either *named* (`sem_open`, shared by name between unrelated processes) or *unnamed*, placed in memory with `sem_init(&s, pshared, value)`; with `pshared` set to 1 and the `sem_t` inside shared memory, related or unrelated processes can use it. `sem_wait`, `sem_trywait`, `sem_timedwait` and `sem_post` operate on it; on Linux they are built on futexes, so an uncontended operation never enters the kernel.

Shared memory is a named object in a tmpfs (`/dev/shm`): `shm_open` returns a descriptor, `ftruncate` gives the object its size, `mmap` with `MAP_SHARED` maps it, and `shm_unlink` removes the name. Its anonymous relative, `memfd_create`, creates such an object with no name at all, reachable only through its descriptor — the mechanism the first engine prototypes used to `dlopen` a shared object received over the network without writing it to disk.

	System V	POSIX
naming	<code>key_t</code> from <code>ftok</code> , or <code>IPC_PRIVATE</code>	<code>/name</code> strings
handle	integer identifier	descriptor (queues, shared memory)
waiting	cannot join <code>poll/epoll</code>	queues can (Linux)
lifetime	until <code>IPC_RMID</code> or reboot	until unlinked and the last user closes
semaphores	sets, atomic multi-operation <code>semop</code> , <code>SEM_UNDO</code>	single counters, futex fast path

2.9 Comparing the mechanisms

The two tables above say what each mechanism *is*. Choosing between them is a question of what each one *costs* per message and what it does for its user — and the two are traded against each other: the less the kernel does per message, the more the processes must do for themselves.

mechanism	copies	syscalls when busy	boundaries	pollable	the other side dies	crosses machines	ma-
pipe, FIFO	2	2	no	yes	EOF; writer gets no EPIPE		
UNIX stream socket	2	2	no	yes	EOF; writer gets no EPIPE		
UNIX datagram socket	2	2	yes	yes	writes fail	no	
TCP, loopback	2	2	no	yes	EOF or reset	yes, unchanged	
POSIX message queue	2	2	yes	yes (Linux)	nothing: the queue stays	no	
System V message queue	2	2	yes	no	nothing: IPC_RMID	until no	
shared memory ring	1 or 0	0	by design	no	nothing, unless perved	su- no	
+ futex/semaphore	—	only sleep/wake	to	no	a robust mutex ports EOWNERDEAD	re- no	
eventfd	—	1	a counter	yes	—	no	
signal	—	1	almost data	no via <code>sigalfd</code>	—	no	

Copies and system calls. Every mechanism that the kernel buffers — pipes, sockets, message queues — copies each message twice, once into the kernel on write and once out of it on read, and costs a system call on each side. A shared-memory ring copies once, straight into pages the reader maps, and a reader that works on the data in place does not copy at all; while the ring

is busy, neither side enters the kernel. That is the whole reason to prefer it, and the reason it scales with message rate where the others scale with system-call cost.

What the kernel no longer does. The copies buy three things that shared memory takes away. *Flow control*: a pipe or socket buffer blocks a writer that is too fast; a ring must track its own free space. *Waiting*: a descriptor can be passed to `poll` or `epoll` with every other descriptor; a ring needs a separate wake-up — a `futex`-based semaphore, or an `eventfd` when it must join an event loop. And *failure*: when a process holding a pipe or a socket dies, the kernel closes its end and the peer reads end-of-file, whereas a process that dies holding a shared ring leaves it exactly as it was — half-written, perhaps, and locked. Robust mutexes (`EOWNERDEAD`) and a supervisor that watches process identifiers are what put that property back by hand.

Boundaries and lifetime. Byte streams have no message boundaries, so the engine frames its own — the length at a fixed offset in every object’s header (Section 8.2) — which is why one reader serves a socket, a pipe and a file alike. Message queues keep boundaries but cap their size (on Linux by default ten messages of at most 8KB per POSIX queue), which rules them out for payloads. System V objects outlive every process until explicitly removed; POSIX objects go when unlinked and unused; descriptors go with their last holder — which is why a shared-memory name is best unlinked as soon as the processes that need it have opened it.

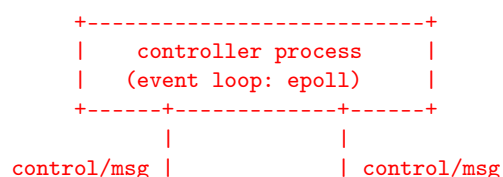
How the engine chooses. Each channel in the engine uses the cheapest mechanism that still gives it the guarantee it needs:

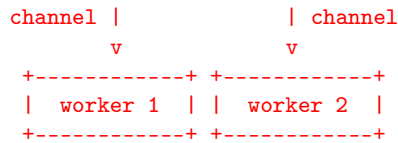
- head → worker: a UNIX stream socket — byte stream framed by the objects, and the head’s death seen by the worker as end-of-file;
- worker → head: a file renamed into place — persistent, atomic, and readable by anyone later;
- a listening socket handed to children: an inherited descriptor across `fork` and `exec` (Section 2.10);
- code received over the network: `memfd_create`, then `dlopen` through its descriptor;
- processes of one node in `engineJavaCore` and `engineLinked`: shared memory with robust mutexes, rings and semaphores, supervised by the controller (Section 9.2);
- between machines: TCP, since nothing else on this list crosses one.

2.10 Controlling many processes from one

A recurring design — and the shape of this thesis’s own node, a head in front of a worker — is a *controller* process that starts workers, hands them work and collects results. It is built in one of two ways.

By message passing. The controller is a server on a central channel — a message queue, a set of UNIX domain sockets or FIFOs — and the workers are its clients. It creates them with `fork`, keeps their state in an array or hash map of structures indexed by `pid_t`, and waits on all of their channels at once with `poll` or `epoll`, so that no single slow worker blocks the rest. A `SIGCHLD` handler, or a `waitpid(..., WNOHANG)` loop, reaps workers as they exit. POSIX message queues suit discrete commands with priorities; `socketpair` or UNIX domain sockets suit full-duplex streams, and can pass descriptors.





By shared memory and semaphores. When throughput or latency matters, the controller places a structure in a shared segment (`shm_open + ftruncate + mmap`, or `shmget + shmat`), with process-shared semaphores inside it:

```

#define MAX_WORKERS 8

typedef enum { STATUS_IDLE, STATUS_RUNNING, STATUS_STOPPING, STATUS_ERROR }
             worker_status_t;

struct ControllerState {
    sem_t lock;
    sem_t work_available;
    struct {
        pid_t      pid;
        worker_status_t status;
        int        command;
        int        result;
    } workers[MAX_WORKERS];
};

```

To assign work, the controller takes `lock`, writes a worker's `command`, updates its `status`, releases `lock` and posts `work_available`; a worker waits on `work_available`, runs the command, writes `result` and signals completion. Only indices and values live in the segment — no pointers — because each process may map it at a different address.

approach	mechanism	synchronisation	best for
datagram / message passing	POSIX or System V message queues	implicit: the queue blocks when full or empty	commands, delegating tasks
stream / multiplexing	UNIX sockets + <code>epoll</code>	explicit non-blocking event loop	flexible IPC, many workers
shared memory	<code>shm_open + mmap</code>	explicit: <code>sem_t</code> , process-shared mutexes	high-volume data, shared state

2.11 Dynamically connected client–server pairs

A server cannot know its clients in advance, yet each client needs a channel of its own. Operating systems solve this the same way wherever it appears: a *well-known rendezvous point* that every client can find, and a *private channel created on demand* for each client that arrives.

```

client A --(1) request + "reply to me here"--> [ well-known point ] <-- server
client B -----> (FIFO path, queue key,
                  listening socket)
               server --(2) creates or opens a private channel per client-->
client A <===== private channel A =====> server (or a worker)
client B <===== private channel B =====> server (or a worker)

```

- **FIFOs.** The server creates a FIFO at a fixed path. Each client creates its own FIFO, named after its process id, and sends a request that includes that id; the server opens the client's FIFO and replies there. The pairing is by naming convention.

- **System V message queues.** The server’s queue has a well-known key; each client creates a private queue with `IPC_PRIVATE` and puts its identifier in the request, and the server replies to it — or, with a single shared reply queue, uses the client’s pid as the message type.
- **Sockets.** The canonical form, and the one the kernel does entirely: the server **listens** on a well-known address, and every **accept** returns a *new* connected socket belonging to one client, while the listening socket goes on accepting others. The private channel is created by the kernel at connection time. `socketpair` produces an already connected pair for a parent and the child it is about to **fork**.
- **Handing a connection on.** A UNIX domain socket can carry an open descriptor to another process (`SCM_RIGHTS`). A broker can therefore accept a client and pass the connected socket to a worker process, pairing the client with the worker dynamically — the model of `inetd` and of socket activation.

Once pairs exist, the server decides how to serve them: *iteratively*, one client at a time; with a process or a thread per client; with a pre-forked pool of workers that take connections in turn; or from a single event loop over non-blocking descriptors with `epoll`. Each trades simplicity against how many clients can make progress at once.

The engine uses these patterns directly. The first prototypes were such pairs — a client and a server per execution strategy, over TCP or UDP. The node’s worker is an *iterative* server on a UNIX domain socket, serving one head connection at a time; the head is a TCP server that accepts a connection from each peer that ships it work; and the alternative deployment of Section 3.5.9, one head and many worker pods, is the controller of Section 2.10 spread over machines, with a connection per worker taking the place of a channel per child.

Chapter 3

Containers, Orchestration, and Scaling

The engine of Chapter 8 is correct but static: a fixed set of worker processes started by hand. To make it scale effortlessly — to add capacity by declaring a number rather than by wiring up machines — its workers must become a fleet that some external system starts, replaces, places, and grows automatically. That system is a container orchestrator [3, 5], and in practice it is Kubernetes running containers built with Docker/OCI tooling. This chapter is a thorough, advanced introduction to both, oriented throughout toward one payoff: run behind an orchestrator, the coordinator/worker engine turns into a *horizontally elastic function service* — tasks submitted, scheduled onto an autoscaled worker pool, and executed on demand. Structurally that is a self-hosted *distributed lambda*, and when tasks form a map/shuffle/reduce DAG it is a *MapReduce*. The recurring theme is a boundary: what the orchestrator may own versus what the engine must never delegate [4].

3.1 Docker and the container model

In one line: a container is a resource-capped, namespaced process — these are enough to simulate an effectively isolated environment.

Historically we could say that containers are the continuation or adaptation of virtual machines, but this is only true in the way that virtual machines are the replacements of actual server machinery. Containers are however the last executable and writable layer on top of an image, in which all the "work" is distilled, and the biggest difference with virtual machines is that containers rely on the host os to run, that is the host kernel, making it easy to have multiple containers in one computer run, and with the isolation mechanisms having been developed them taking turns executing is under complete isolation from each other. Images need to be created by dedicated teams collecting all application dependencies, and it needs to be in need only of one kernel in order to run, because all programs that are to run on the container will be completely separated from the outside world. Otherwise it is an ordinary Linux process (or process tree) started with a restricted view on the host system.

The most important noun is for sure the *image*, which is the read-only basis of the container, just like the java source is the blueprint of a class in which case the container would be an instance of it - an object -, in actuality it would be a running instance of an image with the container being a writable layer on top of it, with what needs to be persisted needing to be attached to a *volume*. The volume (directory) needs to be mount on the container's filesystem.

Relating, the *bind mount*, maps a specific host directory into the container instead, which also facilitates perfectly development needs. Communication to the outside world happens via the docker network drivers.

Containers on Docker's default **bridge** network can reach the outside world, but container-to-container communication does not provide the convenient name-based discovery available on user-defined networks. Consequently, the first practical setup step is almost always to create a

user-defined network, within which containers can refer to one another by name, with Docker providing the necessary DNS resolution.

Docker networks are implemented through different network drivers, each corresponding to a different networking model and use case. The most important drivers are **bridge**, **host**, **none**, **overlay**, and **macvlan**.

The **bridge** driver is the normal choice when several containers run on the same Docker host. Docker creates a virtual network bridge on the host, and containers attached to it receive their own network interfaces and IP addresses. A user-defined bridge is preferable to Docker's automatically created default bridge because it provides automatic DNS-based service discovery and gives the administrator explicit control over which containers belong to the network.

The **host** driver removes much of the isolation of Docker's network. Instead of giving the container its own isolated network stack, the container uses the host's network namespace directly.

The driver **none** provides essentially no network connectivity to the container, which makes it useful for workloads that do not require networking and where network exposure should be minimized.

The driver **overlay** becomes important when containers are distributed across multiple Docker hosts, allowing them to communicate over a shared logical network spanning those hosts.

Finally, **MACvlan** takes a different approach: instead of making the container appear as an endpoint behind Docker's virtual networking, it can give the container its own MAC address and make it appear as a device directly connected to the physical network. This is useful for applications that need to participate directly in an existing Layer-2 network.

Configuration enters through environment variables (and, for anything sensitive, files mounted from tmpfs rather than baked into the image, since image layers are readable by anyone who can pull them). In practice these four are created/defined in a Compose file that declares the images, the volumes, the networks and the published ports for a whole set of services in one place, and brings them up and down together, even defining dependencies between containers.

The most important attribute of the container, and the mechanism by which it can be used and considered an analog to a virtual machine is the mechanisms by which we can enforce its isolation: Namespaces restrict what a process can see (PID, mount, network, IPC, UTS, user, and cgroup namespaces, created by clone with the **CLONE_NEW*** flags), cgroupsv2 limit what it can consume (CPU, memory, I/O, and PIDs), seccomp filters restrict which system calls it may make, and dropped capabilities strip privileges otherwise associated with root, which is a main point of podman, which unlike docker, is daemonless and supports rootless containers, allowing a container to run as root internally while that root identity is mapped through a user namespace to an unprivileged host user, thereby avoiding the need for the container runtime to possess host-root privileges, in linux daemons are all root owned. Namespaces are the mechanism employed in kubernetes as well, they create a folder within the isolated processes have visibility off and the outside world is only available through the managing software and the network. These all show that safety is defined in an application context and for that untrusted code needs to run in a VM-based sandbox (gVisor, Firecracker, Kata).

Three tiers. Docker is not one program but three separable concerns, and almost every property that follows — including every security property — depends on which of the three a mechanism belongs to. The *runtime* is the software that actually runs a container: it talks to the kernel, sets the isolation up, and gets out of the way. The *daemon* sits above it, serving build, start and stop requests and owning the lifecycle of the objects a user names — images, containers, volumes, networks. The *orchestrator* sits above that, deciding where containers run, scheduling them, and reacting to failures. The tiers are defined by interfaces rather than by one vendor's product, which is why each is independently replaceable.

What the runtime builds. A container is not a virtual machine. It is an ordinary Linux process (or process tree) started with a restricted view and a bounded appetite, built entirely from

ordinary kernel primitives. Two mechanisms *construct* it: *namespaces* restrict what it can see — PID, mount, network, IPC, UTS, user, cgroup and time, created by `clone` with the `CLONE_NEW*` flags — and *cgroups v2* bound what it can consume (CPU, memory, I/O, PIDs). Three more *constrain* it once built: dropped *capabilities* strip root’s powers one by one, a *seccomp* filter allows only the syscalls it actually needs, and a mandatory access control profile (AppArmor’s path-based rules, or SELinux’s contexts) confines it at the kernel’s own boundary. The distinction is worth holding on to: the first two make the container, the last three make it safe. There is no guest kernel; a container shares the host’s, which is why it starts in milliseconds and also why, by default, it is an isolation boundary but not a security boundary against a kernel exploit — untrusted code wants a VM-based sandbox (gVisor, Firecracker, Kata).

The runtime’s division of labour. The Docker CLI talks to a daemon that delegates to `containerd`, which starts a *shim* per container, which invokes `runc`. `runc` sets up the namespaces, cgroups and pivot-root, installs the seccomp filter and capability set, and `execves` the entrypoint per the OCI runtime spec — and then exits, leaving nothing of itself behind. The shim is what remains: it becomes the container’s parent, holds its `STDIN/STDOUT` open and reaps it, which is why a container survives a restart of the daemon above it. Because the interface is the runtime spec, the runtime is replaceable (`crun`, `youki`, or a VM-backed implementation), and Kubernetes talks to the same lower layers through the CRI (Container Runtime Interface), so Docker-the-daemon is not required on a cluster — `containerd` suffices.

What the daemon manages: images. Images package the filesystem and metadata, and stand to containers as a class stands to an object. An OCI image is an ordered set of tarball *layers* unioned at runtime (overlays) into a root filesystem, plus a JSON config (entrypoint, env, exposed ports). The daemon does the stacking; what the container sees is a single tree, with one thin writable layer on top that dies with it. Layers are content-addressed, so identical layers are cached and shared across images and pulled once. The practical discipline: order the Dockerfile so rarely-changing layers (dependencies) come before volatile ones (your binary), use *multi-stage* builds to keep compilers out of the runtime image, and ship a *distroless* or `scratch` base — for the statically linked C worker of this project, a few megabytes with almost no attack surface. Reference images by *digest* (`image@sha256:...`) in production, never by mutable tag, so a redeploy cannot silently change the binary.

textbfWhat the daemon manages: images. Images package the filesystem and metadata, and stand to containers as a class stands to an object. An OCI image is an ordered set of tarball *layers* unioned at runtime (overlays) into a root filesystem, plus a JSON config (entrypoint, env, exposed ports). The daemon does the stacking; what the container sees is a single tree, with one thin writable layer on top that dies with it. Layers are content-addressed, so identical layers are cached and shared across images and pulled once. The practical discipline: order the Dockerfile so rarely-changing layers (dependencies) come before volatile ones (your binary), use *multi-stage* builds to keep compilers out of the runtime image, and ship a *distroless* or `scratch` base — for the statically linked C worker of this project, a few megabytes with almost no attack surface. Reference images by *digest* (`image@sha256:...`) in production, never by mutable tag, so a redeploy cannot silently change the binary.

What the daemon manages: networks and volumes. The same tier owns the two objects that outlive a container. Each container receives its own network namespace and a virtual interface whose far end sits on a host bridge, with routing between them and NAT outward; a published port is a rule mapping a host port inward, and it is the one place the isolation is deliberately opened. That default model puts every container on one L2 segment, which is the structural reason inter-container traffic needs separating rather than trusting. Persistent state arrives by mount: a named volume managed by the daemon, a bind mount of a host path, or `tmpfs` for secrets that should never reach a layer. None of it is distributed — two cells on two hosts sharing a volume means a network filesystem, with all of its consistency behaviour intact.

PID 1 and shutdown. A container’s entrypoint runs as PID 1 in its PID namespace, which

changes signal semantics: the kernel delivers no default action for signals to PID 1, and PID 1 must reap orphaned children. So the entrypoint must be the real process (via `exec`), or a tiny init, or `SIGTERM` is ignored and the orchestrator's graceful-stop request is dropped. Correct `SIGTERM` handling — stop accepting work, finish or hand back in-flight tasks, exit — is what turns a worker eviction from a lease-expiry storm into a clean handover, and it is the single highest-value piece of container-awareness the engine needs. Resource limits are cgroup limits: a memory limit yields a deterministic cgroup-OOM kill (a clean, attributable crash), while a CPU limit *throttles* — descheduling the process for the rest of the period — which stalls heartbeats and is the most under-diagnosed cause of false failure detection (watch `cpu.stat`'s `nr_throttled`).

The third tier. Everything above is one host. The orchestrator is the tier that decides which host, keeps the declared number of instances running, and replaces the ones that fail — and because images, networks and volumes are already named, portable objects, it can do so by recreating rather than by moving. That is the subject of the next section, and it is also the boundary of what the container model offers this thesis: relocation at the boundary, not migration of a running computation.

3.2 Kubernetes: orchestration

Mainly kubernetes is an orchestrator for deploying applications, and is generally the de facto infrastructure for distributed systems built today. Kubernetes is a software to manage and deploy applications, literally an orchestrator, the end product is a cluster that is running inside a network of vms, containers, servers. Primarily comprised of two entities (and their constituents): the control-plane and the nodes (data-plane), the management happens declaratively, the desired state is described, and the control is tasked with evaluating the current state, when something is diverging actions are taken to reconcile that. Usually, user-space code will be packaged as a container, and a container will be wrapped in a pod (the smallest deployable unit), next higher-level controllers are going to assist their function across the cores or machines allocated and specified for operation. Although one mostly resorts to distributed architectures for performance gains and reasons, Kubernetes as a manager prides itself in being scalable, reliable and fault tolerant, which are actually the most important aspects for a reliable system, especially with the availability standards that are at play today. What you are buying when you adopt k8s however is efficiency, velocity in deployments, scaling and an abstraction of your infrastructure, which is the opposite of technical debt, as Kelsey Hightower notes in his book. Immutability in the context of Kubernetes is the strategic decision of not updating resources in a way that is not explicitly stated and baked in the configuration or creation logic, instances of an application need to be exactly the same, but it's not possible reliably without an automation layer or the separation of development and deployment that we are able to achieve with docker, Kubernetes and the related technologies. The developer needs to provide solutions that are going to be turned on and off once, they should work with no more than their base arguments and environment, for example updating a library could equally well be achieved by logging in the container and installing the new dependency, or by creating a new one with the updated library baked in, kill the old and launch the new one. Declarative configuration is also easier to understand since it's an 'image' of the world it wants to create, rather than an iteration on a starting point. Kubernetes owned apis and objects are the pods, or group thereof, the actual deployable units that are collections of containers, then services like load balancing, naming and discovery, namespaces that provide isolation the same way docker does for containers and ingress objects that constitute a frontend towards multiple microservices. All these have the effect of isolating the machine from the userspace application that's running, having a negating effect on costs.

Below are the most important control plane components explained:

3.2.1 kubectl / kubeapiserver

What it is: The front door and brain of the control plane. It exposes the Kubernetes API. This is a command-line tool that accesses the kubernetes api, kubectl is just the official image of the client. Typical commands can provide information for the components of a simple cluster like the controller-manager, the scheduler, finally etcd, which is the storage of the cluster.

- **The Only Direct Interface to etcd:** kube-apiserver is the only component in the entire cluster that communicates directly with etcd, while every other control plane component (scheduler, controllers) and worker component (kubelet) has to go through the API server. That means that this is not the writer and that after etcd writes the info and metadata about the state it's only readable in one place, this comes with the extremely important advantage of providing one place to enforce concurrency control for readers and writers on the distributed system that's to be managed.
- **Stateless & Scalable:** Because kube-apiserver stores no local state (it delegates all state persistence to etcd), you can easily scale it horizontally behind a load balancer for High Availability (HA) - installing more instances of it -.
- **Request Pipeline:** Every request, whether issued via kubectl or code, moves through a strict processing pipeline before it is ever persisted, and it takes the form of a verb such as `get`, `list`, `watch`, `create`, `update`, `patch`, or `delete` applied to a resource (or a subresource like `pods/exec` or `scale`).
 1. **Authentication:** Confirms the identity of the caller, using mechanisms like client certificates, bearer tokens, or an external identity provider.
 2. **Authorization:** Checks whether that identity is permitted to perform the requested action, typically enforced through Role-Based Access Control (RBAC).
 3. **Admission Control:** Runs a chain of controllers that can mutate or validate the object on its way in, for example injecting default resource limits or blocking requests that violate security policy.
 4. **Storage:** Finally, the API server writes the resulting state to etcd, persisting the state.

kubectl is the primary tool for interacting with Kubernetes objects, which are organized into namespaces (folders you can target with `--namespace`, or view all of at once with `--all-namespaces`) and accessed through contexts stored in `/.kube/config`, which let you set a default namespace and cluster/user combo via `kubectl config set-context` and `use-context`. To view objects, `kubectl get` lists resources (optionally with `-o wide`, `-o json`, `-o yaml`, or JSON-Path queries via `-o jsonpath` for extracting specific fields), while `kubectl describe` gives a fuller human-readable rundown including related events. Objects themselves are defined as YAML or JSON files and managed declaratively with `kubectl apply -f`, which creates or updates only what's changed (previewable with `--dry-run`) and keeps a history of prior configurations you can inspect with `view-last-applied`; you can also edit objects interactively with `kubectl edit`, or remove them with `kubectl delete`, which executes immediately without confirmation. Finally, labels and annotations — arbitrary tags on objects — are managed with `kubectl label` and `kubectl annotate` (requiring `--overwrite` to replace an existing label, and a trailing `-` on the label name to remove one).

3.2.2 etcd

What it is: A strongly consistent, distributed key-value store based on the Raft consensus algorithm, where functions as the storage of all of the api objects.

- **Cluster Storage of state:** Every single object created in Kubernetes (Pods, Services, CRDs, ConfigMaps) lives inside etcd.
- **Watch Mechanism:** Instead of components constantly polling the database (which degrades performance), etcd supports streaming updates. The API server watches etcd and streams real-time events to the controllers/schedulers.
- **Consistency Over Availability:** etcd prioritizes data consistency over partition tolerance, guarding from 'split-brain' conflicts.

3.2.3 kube-scheduler

What it is: The control plane process responsible for deciding which worker node each newly created Pod should run on, kind of like a scheduler for an OS, but without the preemption.

- **Decoupled Decision Maker:** The scheduler never actually starts a container or pulls an image itself; that responsibility belongs entirely to the kubelet running on the chosen node, which is responsible for 'touching' the pods. It is not able to allocate a position to a node that has just entered the cluster, if we want dynamicity we should use an 'extension' of Kubernetes like the Karpenter or the 'cluster-autoscaler'.
- **Two-Phase Placement Engine:** Placement happens in two distinct passes over the available nodes.
 - **Filtering (Predicates):** The scheduler first eliminates any node that simply cannot host the Pod, for reasons such as insufficient CPU or memory, an incompatible taint, or a missing required node label.
 - **Scoring (Priorities):** Among the nodes that remain, it then ranks each one by fit — for example how evenly workloads are spread across zones, proximity to needed data, or overall resource efficiency — and assigns the Pod to whichever node scores highest. Mainly used for reallocating pods after failures or on scaling.

3.2.4 Cluster Autoscaler / Karpenter

What it is: A component that runs *alongside* the control plane, rather than as one of its core pieces, whose job is to add or remove worker nodes based on demand, not part of the default installation of Kubernetes.

- **The Trigger:** Both tools watch the API server for Pods stuck in the **Pending** state because no current node can satisfy their resource requests, and treat that backlog as a signal that more capacity is needed.
- **Cluster Autoscaler:** The older, more established of the two. It works in terms of pre-defined node groups (an AWS Auto Scaling Group, a GCP Managed Instance Group, etc.) and simply scales those groups up or down, meaning new nodes come in whatever instance size and shape the group was already configured with.
- **Karpenter:** A newer, more flexible alternative that provisions individual nodes directly against the cloud provider's API, without needing pre-defined node groups. It picks the specific instance type, size, and even spot-vs-on-demand mix that best fits the exact Pods waiting to be scheduled, which typically means faster and more tightly bin-packed provisioning.
- **Scaling Down:** Both tools also work in reverse, continuously looking for nodes that are underutilized or empty and safely draining and terminating them to avoid paying for idle capacity.

- **Division of Labor:** Once a new node registers with the API server and is marked `Ready` (via the Node Controller in `cloud-controller-manager`), it becomes just another candidate node, and it is `kube-scheduler` — not the autoscaler — that decides which of the pending Pods actually land on it.

Finally it should be noted that the constituents of a cluster are in and of themselves deployed via kubernetes, kubernetes DNS provides naming and discovery for the services defined, also a replicated service, additionally there is the kubernetes proxy that's responsible for load balancing and traffic routing over the network of the cluster.

The most important moving parts are the data plane nodes:

3.2.5 Worker Node

What it is: Every worker node runs the same three-piece stack: an agent, a runtime, and a network proxy.

Worker Node

```
|-- kubelet
|-- Container Runtime
|-- kube-proxy
```

3.2.6 kubelet

What it is: The main Kubernetes agent running on every worker node.

- Communicates with the `kube-apiserver` to learn what should be running on its node.
- Receives Pod specifications describing the containers, images, and resources a Pod needs.
- Ensures the assigned Pods are running by starting their containers via the container runtime.
- Monitors containers to confirm they stay in the expected state.
- Reports node and Pod status back to the control plane so it always has an accurate picture.
- Performs health checks and restarts containers when necessary, enabling self-healing.

3.2.7 Container Runtime

What it is: The container runtime is responsible for actually running the containers.

Examples include:

- `containerd` – the most widely used runtime, and the default in most managed Kubernetes offerings.
- `CRI-O` – a lightweight runtime built specifically for Kubernetes.

The kubelet communicates with the runtime through the Container Runtime Interface (CRI), so any compliant runtime can be swapped in without changing kubelet itself.

3.2.8 kube-proxy

What it is: `kube-proxy` provides the networking functionality required for Kubernetes Services.

- Maintains network rules on the node, typically using `iptables` or `IPVS`.
- Enables communication with Services by resolving their stable virtual IP to real Pod addresses.
- Routes traffic to the appropriate Pods, load-balancing across all healthy backends.

- Implements Service types such as **ClusterIP** and **NodePort**, giving each a working network behavior.

What it is: Every kind of Kubernetes object shares the same `apiVersion/kind/metadata/spec/status` shape and the same declarative contract — you write `spec`, Kubernetes writes `status`, and a controller continuously reconciles the two. What differs from one kind to the next is what that controller actually *does*: a Pod just runs containers, a Deployment manages rolling updates, a Job runs something to completion. The rest of this section is a reference card for each of the kinds you'll use most, showing how they relate to one another, what operations are available on each, and a minimal YAML example.

3.2.9 How the Kinds Relate

- **Deployment** owns a **ReplicaSet**, which owns **Pods**.
- **CronJob** owns a **Job**, which owns **Pods**.
- **DaemonSet** owns **Pods** (one per node).
- **StatefulSet** owns **Pods** (stable identity and storage).
- **Service** selects **Pods** — it does not own them.
- **ConfigMap** and **Secret** are mounted into **Pods** — again, no ownership.

Solid arrow = owns / creates (garbage-collected together via `ownerReferences`). **Dashed arrow** = selects or mounts into (no ownership, no cascading delete).

3.2.10 Pod

What it is: The smallest deployable unit in Kubernetes — one or more containers that share a network namespace and storage volumes, and are always scheduled together on the same node. Pods are described in a manifest, that describes them to kubernetes since they are the core object, pods can consist of multiple containers or can host one and communicate over the network, each one is controlled by a cgroup for resource limits and will share the appropriate namespaces with neighboring, not identical pods. Most important design decision for splitting the application's components into pods is what scaling needs are there for each, since a pod is the equivalent of an atom for kubernetes. Kubectl will accept and store the manifests in etcd, then the scheduler will take over, and will ensure if possible to distribute the pods onto different machines for reliability.

You can:

- **Create** — `kubectl apply -f pod.yaml → POST /api/v1/namespaces/{ns}/pods`.
- **Inspect** — `kubectl get/describe pod → GET` the Pod resource or its `status` subresource.
- **Exec into it** — `kubectl exec → POST .../pods/{name}/exec`.
- **Read logs** — `kubectl logs → GET .../pods/{name}/log`.
- **Port-forward** — `kubectl port-forward → POST .../pods/{name}/portforward`.
- **Kill it** — `kubectl delete pod → DELETE .../pods/{name}`. If it's owned by a **ReplicaSet**, **DaemonSet**, or **StatefulSet**, a replacement appears automatically.
- **Cannot** scale, roll back, or migrate itself — Pods are disposable by design; those behaviors belong to the controllers above it. Kubectl exposes a basic api that's similar to docker for inspecting pods and scheduling them, that's because they can be almost identical in content.

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80

```

3.2.11 ReplicaSet

What it is: A controller that guarantees a fixed number of identical Pods are running at all times, matching Pods purely by label selector rather than by name. These provide redundancy, scaling and sharding, and forms the basis for the reliability and fault tolerance of Kubernetes. It's designed to group pods together and expose them as one, management takes place via reconciliation loops. This is the first instance of the configuration of the desired state defined earlier. What's being stated is the number of pods and their configurations, mechanisms have been defined to 'reconcile' each and every discrepancy from the blueprint. Labels are used to mount a pod to a replica set, also replicas are not coupled with the services mentioned later in this thesis about service discovery, dns and load balancing because it goes against the loose coupling principles of Kubernetes.

You can:

- **Create** — `kubectl apply -f rs.yaml → POST .../replicasets`.
- **Scale** — `kubectl scale rs --replicas=N → PATCH .../replicasets/{name}/scale`.
- **Delete** — `kubectl delete rs → cascades to its owned Pods via ownerReferences`.
- **In practice** you rarely create these directly — a Deployment creates and manages them for you.

```

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27

```

3.2.12 Deployment

What it is: The standard controller for stateless applications, managing ReplicaSets on your behalf so you get rolling updates, rollbacks, and self-healing without touching Pods directly.

You can:

- **Run / create** — `kubectl create deployment` or `apply -f deploy.yaml` → POST `.../deployments`.
- **Scale** — `kubectl scale deployment -replicas=N` → PATCH `.../deployments/{name}/scale`.
- **Update (rolling deploy)** — `kubectl set image deployment/x c=img:v2` → PATCH `spec.template`; a new ReplicaSet is created and scaled up while the old one scales down.
- **Roll back** — `kubectl rollout undo deployment/x` → reverts to a previous ReplicaSet revision.
- **Pause / resume a rollout** — `kubectl rollout pause | resume`.
- **Restart** — `kubectl rollout restart` → patches a pod-template annotation, forcing fresh Pods even with no spec change.
- **Stop / kill** — `kubectl delete deployment` → DELETE `...`; cascades to its ReplicaSets and their Pods.
- **“Migrate”** — Kubernetes never live-migrates a running Pod. Moving work off a node means cordoning it (`kubectl cordon`) and draining it (`kubectl drain`), which evicts its Pods so the Deployment’s controller reschedules replacements elsewhere.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

3.2.13 StatefulSet

What it is: Like a Deployment, but for workloads that need a stable, unique network identity and stable per-Pod storage that survives rescheduling — think databases and other clustered stateful apps.

You can:

- **Create / scale / delete** — same commands as a Deployment, but Pods are created, scaled, and terminated in order (`web-0`, `web-1`, `web-2`, ..., reverse order on scale-down).
- **Update** — `spec.updateStrategy` is either `RollingUpdate` (default, ordered) or `OnDelete` (you delete Pods manually to trigger their replacement).
- **Stable identity** — each Pod keeps the same hostname and, via `volumeClaimTemplates`, the same `PersistentVolumeClaim` even after being rescheduled to a different node.
- **Delete** — `kubectl delete statefulset` removes the Pods but, by default, leaves the PVCs (and their data) behind.

```

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "web"
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi

```

3.2.14 DaemonSet

What it is: A controller that ensures exactly one copy of a Pod runs on every node (or every node matching a selector) — the pattern used for log collectors, node monitoring agents, and network plugins. Typically these are the equivalent of the daemon, background running tasks that are present in oses, and accordingly they have monitoring roles, and this is achieved by replicating one node to each node, installing in this way a daemon.

You can:

- **Create** — `kubectl apply -f ds.yaml → POST .../daemonsets`.
- **“Scale”** — not by replica count; the Pod count tracks the number of matching nodes automatically, so it grows or shrinks as nodes join or leave.
- **Update** — `RollingUpdate` (default) or `OnDelete`, same idea as `StatefulSet`.
- **Delete** — `kubectl delete daemonset` removes the Pod from every node.

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter

```

Hightower states: “ReplicaSets need to be run when the application is completely decoupled from the node you are using, while DaemonSets are for architectures where pods with your whole application must be present on all nodes of the cluster”. There is however a way to specify a node subset for specific deployments, labels can be set as selectors for those cases.

3.2.15 Job

What it is: A controller for run-to-completion workloads — it tracks successful completions rather than keeping Pods alive forever, and retries failed Pods up to a limit. A job is the creation of a pod that runs until termination, batch jobs, database migrations and things that need to happen once and completely are the scope.

You can:

- **Run** — `kubectl create job` or `apply -f job.yaml → POST .../jobs`.
- **Control throughput** — `spec.completions` and `spec.parallelism` decide how many Pods must succeed and how many can run at once.
- **Suspend / resume** — `kubectl patch job x -p '{"spec":{"suspend":true}}'` pauses it without deleting it.
- **Stop / kill** — `kubectl delete job -cascade=foreground|orphan` controls whether its Pods are deleted along with it.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  completions: 1
  parallelism: 1
  template:
    spec:
      containers:
        - name: migrate
          image: myapp/migrate
      restartPolicy: Never
```

Once again jobs are managing and deploying pods to get the job done, and in so doing we can achieve parallelization of workloads and even define work queues.

3.2.16 CronJob

What it is: A scheduler that creates a new Job on a cron-style schedule — the Kubernetes equivalent of a crontab entry, but producing real, trackable Job/Pod objects each run.

You can:

- **Create** — `kubectl create cronjob` or `apply -f cronjob.yaml → POST .../cronjobs`.
- **Suspend** — `kubectl patch cronjob x -p '{"spec":{"suspend":true}}'` stops new runs without deleting the schedule.
- **Control overlap** — `spec.concurrencyPolicy: Allow, Forbid, or Replace`.
- **Trigger manually** — `kubectl create job -from=cronjob/x manual-run-1` runs it immediately, outside the schedule.
- **Delete** — `kubectl delete cronjob` removes the schedule and its recent Jobs.

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: myapp/backup
              restartPolicy: OnFailure

```

3.2.17 Service

What it is: A stable virtual address that load-balances traffic to whichever Pods currently match its label selector, so clients never need to track individual Pod IPs.

You can:

- **Create** — `kubectl expose deployment/x -port=80` or `apply -f svc.yaml → POST .../services`.
- **Change routing** — editing `spec.selector` or `spec.ports` re-routes live traffic as soon as kube-proxy picks up the updated Endpoints, no Pod restart needed.
- **Choose exposure** — `spec.type`: `ClusterIP` (internal only), `NodePort` (also exposed on every node), or `LoadBalancer` (provisions a cloud load balancer via `cloud-controller-manager`).
- **Delete** — `kubectl delete service` removes the virtual IP/load balancer; it never touches the backing Pods themselves.

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP

```

Generally, service discovery is a contiguous problem, and abstracts/solves the discovery - literally- of the actual pod that's used to implement the actual service, even a routing table can be used, but in this case specifically we care about dynamicity of this since there are multiple levels of the stack from os to hardware to pod and to container that may be changing. This is why traditional DNS fails, SRV records are an alternative, however going further more suitable to the specific domain and application routing needs to be set up, perhaps with specialized or custom software. For outwards port forwarding there is the nodeport feature that's designed for allowing traffic in one namespace, by being able to reach just one node of the cluster.

3.2.18 ConfigMap & Secret

What it is: Two nearly identical objects for injecting configuration into Pods without baking it into the container image — a `ConfigMap` for plain values, a `Secret` for sensitive ones (base64-encoded at rest, and typically further protected by encryption-at-rest and RBAC).

You can:

- **Create** — `kubectl create configmap/secret` or `apply -f` → `POST .../configmaps` or `.../secrets`.
- **Consume** — mount as a volume (files appear under a directory) or inject as environment variables via `envFrom/valueFrom`.
- **Update** — `kubectl edit` or `apply` → `PATCH ...`; volume-mounted files update on the kubelet's next sync automatically, but env-var injections require the Pod to restart to pick up the change.
- **Delete** — `kubectl delete configmap/secret`; Pods already using the old data keep running until they're restarted.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQ= # base64-encoded
```

3.3 The object model

, and where each fits the engine: This what the context of kubernetes encompasses, defined below are the nouns and verbs of kubernetes, making it clear what actions and objects we can handle for the management of the lifecycle of our userspace programs.

Pod	one+ co-scheduled containers sharing net+IPC ns (localhost, sidecars)	
Deployment	N interchangeable STATELESS pods, rolling updates	-> WORKERS
StatefulSet	stable names/storage, ordered ops	-> COORDINATOR quorum
Job / CronJob	run-to-completion / scheduled batch	-> occasional batch only
DaemonSet	one pod per node	-> node agents
Service	stable virtual IP + L4 LB over a pod set	
Headless Svc	one DNS A-record per pod (no VIP)	-> peer discovery
ConfigMap/Secret	configuration / credentials	
PVC / PV / StorageClass	durable volumes via CSI	-> logs, DB state
Ingress / Gateway	north-south HTTP entry	-> Submit front door
HPA / VPA	autoscale replicas / right-size requests	-> scale workers
PodDisruptionBudget	cap voluntary disruption	-> protect quorum
NetworkPolicy / RBAC / SA / securityContext	least privilege	
CRD + controller	extend the API with your own object + reconcile loop	

Buy consensus; do not autoscale it. A quorum's size is a correctness parameter, not a capacity knob — an HPA must never resize the coordinator StatefulSet, because adding a member is a membership-change protocol (Chapter 8), not a replica bump. And run *your own* etcd: sharing the cluster's control-plane etcd couples your load to the API server's health, a documented way to break a cluster.

Durability on Kubernetes has its own caveats: a PVC is not a backup (snapshot to object storage and *test restore*); the log's device must give low-latency `fsync`, so prefer local NVMe over network storage whose sync is a network round trip.

3.4 API gateways, ingress, and service mesh

Traffic splits into *north-south* (clients → cluster) and *east-west* (service → service), and each has an edge.

North-south is the Ingress / Gateway API and its controller (Envoy, NGINX, Traefik): one entry point that terminates TLS, routes by host/path, enforces authentication and rate limits, and splits traffic for canary or blue-green rollouts. For the engine this is the *Submit* front door: it authenticates callers, applies per-tenant rate limits and load shedding (Chapter 8), and routes to the coordinator leader — so the admission and backpressure logic lives at a single, observable choke point rather than being scattered.

East-west is optionally a *service mesh*: a sidecar proxy (Envoy) injected beside every pod that transparently adds mutual TLS (service identity), retries, timeouts, circuit breaking, and per-call telemetry. This is exactly the fault-tolerance and RPC-hardening machinery of Chapter 8, *externalized* out of the application into the platform. The trade is real overhead (a proxy hop and CPU per pod) and operational complexity, so a mesh earns its place at scale or across trust boundaries, not by default; a C engine that already implements deadlines, retries with budgets, and mTLS in its own RPC layer may reasonably skip it and keep the control it already has.

3.5 The engine on Kubernetes: which entities it uses, and for what

The sections above describe the entities in general. This one maps them onto the engine as it is actually deployed (`engineCJavaKubernetes`, `deploy/k8s/`): two ordinary PCs joined by a 1 Gb cable and running k3s, one image (`engine:dev`) for every role, and the engine of Chapter 8 otherwise unchanged. Kubernetes contributes placement, restarts, stable names and the number of nodes; it does not take over any decision the engine already makes.

entity	object	role in the engine
Namespace	<code>engine</code>	everything below; deleting it removes the deployment
Pod	<code>engine-K</code>	one engine node: a C worker and a Java head
StatefulSet	<code>engine</code>	the nodes, with stable names
StatefulSet	<code>etcd</code>	the cluster layer's store (Section 8.5)
Service	<code>engine</code> (headless), <code>etcd</code>	one DNS name per node; one for the store
Job	<code>submit</code>	one batch of work, then it exits
ConfigMap	<code>engine-config</code>	threads, slots, what to submit
ScaledObject	<code>engine</code> (KEDA)	the number of engine pods, from worker CPU

The current deployment. The engine as it stands now (Section 8.6) is deployed by a separate project, `engineKubernetes`, built from the two projects beside it: `engineJava` for the engine and `pipeIn` for the work. It keeps the central decision of the earlier deployment — one pod is one engine node, a worker and a head sharing a directory, resolving artifacts and running jobs exactly as they do on a machine — and changes what surrounds it:

- two machines on an ordinary LAN instead of a dedicated cable;
- exactly two engine pods, one per machine, instead of an autoscaled number;
- no submit Job: work is pushed to the heads, and results fetched from them, over the network, by pipeIn speaking the engine's own protocol;
- two NodePort Services, so that any machine on the LAN can do that.

entity	object	role in the current deployment
StatefulSet	engine , 2 replicas	one node per machine (required anti-affinity)
StatefulSet	etcd	unchanged: the heads' live claims
Service	engine (headless)	one DNS name per node, which each head advertises
Service	engine-0-lan , engine-1-lan	NodePorts 30100 and 30101: each head, reachable from the LAN
ConfigMap	engine-config	etcd's address, worker threads, advertised slots

It is built and checked — the manifests parse and agree with each other on every shared path, variable, port and selector, and the image is built from sources alone — but at the time of writing it has not yet been run on the two machines, and no result is claimed for it.

3.5.1 Pod: one pod is one engine node

The head and the worker talk over a unix socket and reply through the filesystem (Section 8.4), so they must share a machine and a directory. That is exactly what a Pod guarantees: two containers — the worker (`/opt/engine/node`) and the head (`JobServerCli` on port 9100) — scheduled together, sharing an `emptyDir` volume mounted at `/data` that holds `engine.sock`, `functions/` and `datasets/`. Each container asks for CPU (worker 1, head 0.1): the scheduler packs pods by these requests and the autoscaler measures against them. The pod is Ready when the worker's socket exists and the head is listening. The `emptyDir` makes the lifetime explicit: a node's results live exactly as long as its pod.

In the current deployment the pod is the same pair, with two changes that follow from the engine itself. The worker now serves exactly one head for its whole life and removes its socket once that head connects, so “the socket exists” is true only *before* the node is working and can no longer be a readiness test; the pod is Ready when the head is listening, which it does only after it has reached its worker. And because both containers start at once, the head retries the worker's socket for up to thirty seconds rather than failing on the first attempt; if it later loses the worker it exits, and the kubelet restarts whichever container stopped — the worker, too, exits when its one head goes. The node therefore restarts as a pair without any ordering in the manifest.

3.5.2 StatefulSet rather than Deployment

A Deployment's pods are interchangeable and renamed on every restart. An engine node is not interchangeable: it advertises an address in etcd that other nodes dial. A StatefulSet gives every pod a stable ordinal name (`engine-0`, `engine-1`, ...) that survives restarts, which is why it is used for the nodes even though they hold no durable state. Two properties follow and both matter. Pods are created in parallel (`podManagementPolicy: Parallel`), so scaling up is not serialised; and scaling down always removes the highest-numbered pods, busy or not — taking the jobs queued on them and the results in their `emptyDir` with them. A topology spread constraint spreads the pods evenly over the two PCs.

In the current deployment the StatefulSet has exactly two replicas, and a *required* anti-affinity on the host name replaces the spread constraint: two engine pods may never share a machine. With both machines up, `engine-0` and `engine-1` land one on each; with one, the second pod stays Pending — a visible signal rather than two nodes quietly doubled up on one machine, where they would read the same load and hide the point of the exercise.

The store runs as a second, separate StatefulSet (`etcd`), one member on an `emptyDir`. It is the engine's own etcd, not the cluster's control-plane store, as recommended above; losing its data costs nothing, because every key is a leased live claim that the heads republish within a second.

3.5.3 Service: headless, for naming only

The `engine` Service is headless (`clusterIP: None`): it provides one DNS record per pod, `engine-K.engine.engine.svc.cluster.local:9100`, and no load balancing. Each head advertises that name in etcd, and the submitter dials it. This is deliberate: which node receives a job is the engine’s decision (round robin over the nodes etcd lists as live), so a Service that picked a pod on its own would contradict the placement rule. The `etcd` Service, by contrast, is an ordinary one — a single stable address for the store.

In the current deployment the headless Service is used between the heads: each advertises `engine-K.engine.engine.svc.cluster.local:9100` in etcd, and a head that needs a `.so` or a dataset looks the other one up there and dials it directly. Two more Services open the heads to the LAN, one *per pod*: `engine-0-lan` selects the pod named `engine-0` and exposes it on port 30100 of every machine, `engine-1-lan` does the same on 30101. They are NodePorts rather than one balanced Service for the same reason as above: which node receives a job must be decided by whoever sends it, not by a load balancer. The engine has no authentication, so these ports open “run this code” to the whole LAN — acceptable on a home network, not on a shared one.

3.5.4 Job: a batch, submitted and gone

A submission is run-to-completion work, so it is a Job. Its pod runs three things in order: an init container that builds the batch (`jobs.bin` plus the function and the datasets it names); a worker as a *native sidecar* (an init container with `restartPolicy: Always`, Kubernetes 1.29+), because the head refuses to start without a local worker; and the head in file mode, `JobFileCli --spread --slots 0`. With zero slots the submitter runs none of the jobs itself and ships every one, with its function and data, to the engine pods — necessary, because a Job’s pod and its disk disappear when it completes, and anything computed there would be lost.

In the current deployment there is no submit Job. A head listens on TCP and speaks the engine’s protocol to anyone, so `pipeIn`, from any machine on the LAN, pushes the job file to the heads (`send`) and asks them for the results by job identifier (`fetch`): a `REQUEST` for the `DATA` named by the job, answered with the `DATA` or with `NOT_FOUND`, repeated until every result is in and then compared byte for byte with the answers `pipeIn` computes itself. With both nodes given, jobs alternate between them, so the dependent chains cross machines: a head whose job depends on a result produced on the other node asks that node’s head for it, sweep after sweep, until it exists. A head addresses such requests only to another head — given on the command line or listed in etcd — and never to a client that connected to push work. The functions are compiled inside the image, by the compiler whose output the workers will load, and the job file is taken from the image; a job built on the submitting machine would name different bytes, and a different identifier.

3.5.5 ConfigMap and the downward API

`engine-config` holds the tunables — etcd’s address, worker threads, advertised slots, and what the submit Job generates. Containers receive it as environment variables, and Kubernetes substitutes them into the command line (`--threads $(ENGINE_THREADS)`). A pod’s own name, namespace and IP come from the downward API (`fieldRef`), which is how a head knows its node identifier and the address to advertise without any configuration per pod. A changed ConfigMap reaches running pods only on a restart, which the deployment script performs.

3.5.6 Autoscaling: KEDA, the HPA and metrics-server

KEDA is set up in three steps, all driven by `deploy/k8s.sh keda`. `metrics-server` comes first, because the CPU trigger is an HPA underneath and the HPA reads pod CPU from it; `k3s` ships it, and on other clusters the script installs it. KEDA itself is then installed into its own `keda` namespace from its Helm chart (or its release manifest, when Helm is absent), and the script waits until all three of its components are available — the operator, its metrics API server, and its admission webhook, which refuses any `ScaledObject` until it answers. Last comes the `ScaledObject`, `deploy/k8s/autoscale/engine-scaledobject.yaml`, kept outside the main manifests because its kind exists only once KEDA's CRDs do. It names the `engine` `StatefulSet` as its target, keeps between 2 and a configurable maximum of pods (`KEDA_MAX`, about the two machines' cores together), triggers on the `worker` container's CPU at a 70 % target of its 1-CPU request, and shapes the HPA's behaviour: at most two new pods every 30s on the way up, and on the way down ten quiet minutes, then one pod every two minutes. `deploy/k8s.sh keda-off` deletes it and returns the `StatefulSet` to a fixed size.

The number of engine pods is managed by a KEDA `ScaledObject`, which KEDA turns into a Horizontal Pod Autoscaler. Its trigger is the CPU of the `worker` container only — the head idles — read through `metrics-server`, which `k3s` ships. The HPA computes $\text{desired} = \lceil n \cdot u / \text{target} \rceil$ for n pods of average utilisation u . With one worker thread per pod, a busy pod reads at most 100 %, so one step can grow the cluster by at most $100/\text{target}$: at a 70 % target the cluster crept from 2 to 3 pods and stayed there, because newly started pods idle until the next batch and pull the average below the target; at 25 % it grew from 2 to 8 pods in about 70s. In keeping with the rule above, only the engine `StatefulSet` is autoscaled — never `etcd`.

Scaling interacts with the push model. A job is bound to the pod it was sent to, so a new pod receives work only from the next batch, and scale-down would remove pods that still hold queued jobs. The stress test therefore disables scale-down until every job has a result, and the deployment script pauses the `ScaledObject` while it restarts the pods, so the two controllers do not fight over the replica count.

In the current deployment autoscaling is not used: the aim is two machines, one node each, and a fixed pair is what makes cross-machine behaviour observable. Nothing in the engine prevents it — a new pod publishes itself to `etcd` and becomes a head the others can reach — but under the push model it would receive work only from the next submission, as described above.

3.5.7 Scheduler, kubelet and runtime, and the network

These are used as they come. The scheduler filters nodes by the CPU the pods request: at 1.1 CPU per engine pod, the two PCs hold 35 of them (14 and 21). Each kubelet starts containers through `k3s's containerd`, which pulls `engine:dev` from an image registry on the first PC. Pod traffic crosses between the machines as flannel VXLAN, pinned to the cable (`--flannel-iface`), so a job shipped to a pod on the other PC travels over the direct link.

In the current deployment the machines share an ordinary LAN, and flannel is pinned to each machine's LAN interface instead. Two scripts set the cluster up: one on the first machine installs the `k3s` server and an image registry and opens the ports the machines need between them (6443 for the API, 8472/udp for flannel, 10250 for the kubelet) and those the LAN needs (30100 and 30101); one on the second machine joins it as an agent that pulls the engine image from the first. Both engine pods therefore run the same image, built once.

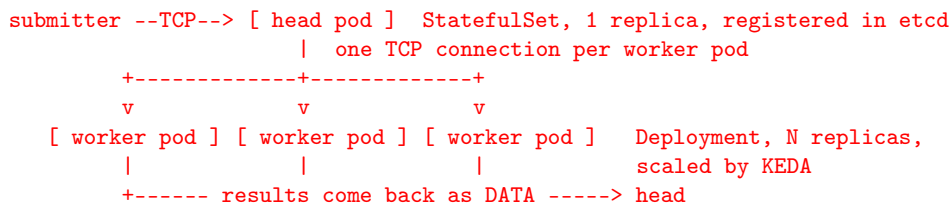
3.5.8 What is deliberately not used

- **Deployment, ReplicaSet** — nodes need stable names; see above.

- **DaemonSet** — one node per machine would forbid scaling by pods.
- **PersistentVolumeClaim** — results are on `emptyDir`; a claim per pod (volumeClaimTemplates) is the next step if results must outlive scale-down.
- **Ingress, Secret, CronJob** — no external clients, no credentials, no schedule yet.
- **Cluster Autoscaler, Karpenter** — the machines are two fixed PCs; KEDA scales pods within them, and a Pending pod is the signal that they are full.

3.5.9 An alternative split: one head pod, many worker pods

The deployment above keeps the node of Section 8.4 intact: every pod is a head and a worker, and the engine scales by whole nodes. The other natural split puts a single head in one pod and the workers in many pods of their own — a coordinator with executors instead of a set of peers.



What has to change in the engine. The present node relies on two things that only hold inside one pod. The worker listens on a unix socket, so it would need to accept TCP as well; the framing needs no change, since the same objects are valid on any byte stream (Section 8.2), and the head can already write to a `host:port` — it does so when it ships work to a peer. The harder change is the reply path: a worker that shares no directory with the head cannot answer by writing a file. The natural answer within the object model is to send the result back on the same connection as an ordinary DATA object, which the head stores and publishes as it does today. That gives the protocol its first reply, and with it something the engine currently lacks: the head knows which jobs each worker still holds.

How it would be deployed.

- **Workers: a Deployment**, not a StatefulSet. A worker no longer advertises an address that anyone else dials by name — only the head talks to it — so its identity does not matter and interchangeable replicas are the right model. Each pod is the C worker alone, one thread, its function cache and datasets on an `emptyDir`.
- **Discovery: a headless Service** over the worker pods. Its DNS name resolves to every ready worker’s address, and the head re-resolves it periodically to pick up pods that KEDA has added. An ordinary Service would be wrong here: it balances *connections*, not jobs, and the head holds one long-lived connection per worker, so placement must stay the head’s decision.
- **Head: a StatefulSet of one replica** with a stable name, reached by submitters through an ordinary Service and registered in etcd as the single node. One head is also one owner of every job, which keeps the single-writer argument of Section 7.6 trivially true. Its catalogue wants a PersistentVolumeClaim, since it now holds every input and every result.
- **Autoscaling: KEDA on the worker Deployment only**, with the same CPU trigger. A new worker receives jobs as soon as the head opens a connection to it — not from the next batch, as in the present design — because the head, not a submitter, decides where each job goes.
- **Scale-down** still removes pods that may hold queued jobs, but no longer silently: the head sees the connection close and re-sends that worker’s unfinished jobs elsewhere. A `preStop` hook and a termination grace period let a worker finish what it holds before it goes.

What it costs. Every job’s data now crosses the network from the head to a worker, where today a node runs work on data it already holds, over a local socket — the head’s link becomes the throughput limit, and with the two PCs, half the workers sit across the cable from it. The

head is also a single point of failure for the whole cluster, where today losing a node loses only that node. What the split buys is workers that are stateless, interchangeable and cheap to add or remove; failures that are detected rather than silent; and one place that knows the state of every job. It is, in effect, the pull design's coordinator reached by a shorter step: keep the head's catalogue, give the worker a TCP listener and a reply, and let the head do the distribution a queue would otherwise do.

Chapter 4

Two Transports: XDP and gRPC

As we see on the implementation of the rudimentary engine, computing costs across machines that are in the same network and not necessarily in close proximity are a detriment to performance.

BPF originally emerged as a technology for efficiently filtering network packets. When you run `tcpdump` with an expression—such as matching a specific host or port—the expression is compiled into optimized BPF bytecode and executed by a sandboxed virtual machine inside the kernel.

Over time, BPF evolved significantly beyond its original purpose of packet filtering. With the introduction of **extended BPF (eBPF)**, the virtual machine gained the ability to execute custom programs in response to many different events occurring within the Linux system. Instead of only examining network packets, eBPF programs can be attached to system calls, kernel functions, tracepoints, network events, and application-level probes.

This makes BPF a powerful mechanism for **observability, tracing, performance analysis, networking, and security**. For example, an eBPF program can monitor system calls, measure how long processes spend waiting for CPU time, trace file-system operations, inspect network activity, or capture stack traces when applications block. These programs run in the kernel while being checked by the BPF verifier, which ensures that they meet safety requirements before they are loaded.

Developers typically interact with BPF through higher-level tools and frameworks such as **BCC** and **bpftool**. BCC is commonly used for building more complex tracing tools and applications, while bpftool provides a simpler interface for writing short tracing programs and one-liners. Together, these tools make it possible to analyze and understand Linux systems without modifying the kernel or adding traditional instrumentation to every application.

Although the technology is formally known as **eBPF** to distinguish it from the original Berkeley Packet Filter, it is now commonly referred to simply as **BPF**. In modern Linux systems, BPF provides a general-purpose way to safely extend kernel functionality and observe system behavior dynamically.

The control-plane/data-plane split that runs through this thesis (Chapter 8) has a transport at each end, and they pull in opposite directions. The *data plane* — bulk task payloads, results, checkpoint images moving between nodes — wants the lowest possible per-byte and per-packet cost, even at the price of building reliability by hand. The *control plane* — **Submit**, dispatch, status, the worker channel — wants typed, evolvable, reliable request/response with deadlines and streaming, and to fit the Kubernetes ecosystem of Chapter 3. This short chapter presents one technology aimed squarely at each: **XDP** for the hot data path in the kernel, and **gRPC** for the application-layer control path.

4.1 XDP: programmable packet processing at the driver

XDP (eXpress Data Path) is a high-performance networking technology built on top of BPF/eBPF. It allows BPF programs to run at a very early stage of packet processing, immediately after a network packet is received by the network interface and before it passes through most of the Linux networking stack. Because the packet can be processed so early, XDP can make decisions with very low overhead, which makes it particularly useful for high-speed packet processing.

An XDP program can inspect the contents of an incoming packet and decide what should happen to it. For example, it can drop unwanted packets, pass packets to the normal Linux networking stack, redirect packets to another network interface or CPU, or transmit a packet directly back out through the interface. This makes XDP useful for applications such as DDoS protection, firewalls, load balancing, packet filtering, and high-performance networking. XDP (eXpress Data Path) [7, 6] runs an *eBPF* program at the earliest point in the receive path — inside the NIC driver, before the kernel allocates an `sk_buff`, and on capable hardware offloaded onto the NIC itself. eBPF is sandboxed bytecode: the kernel *verifier* proves the program terminates and touches only permitted memory, then JITs it to native code, so it runs at kernel speed without the ability to crash the kernel; persistent state lives in *maps* shared with userspace. An XDP program sees each raw frame and returns a verdict: `XDP_DROP` (discard — the basis of line-rate DDoS filtering), `XDP_PASS` (hand up to the normal stack), `XDP_TX` (bounce back out the same NIC), or `XDP_REDIRECT` (send to another NIC, another CPU, or a userspace socket). Because it acts before the sockets/TCP machinery exists, it is where an L4 load balancer (spreading `Submit` or dispatch across coordinators and workers) or a custom router belongs when the per-packet cost of the full stack is the bottleneck. The key idea is that XDP is not a separate replacement for BPF. Instead, it is one of the mechanisms through which BPF programs can be attached to the Linux networking path. While traditional BPF tracing can observe many parts of the system, XDP specifically focuses on processing network packets as early and efficiently as possible.

The piece most relevant to this project’s data plane is **AF_XDP**: a socket type that, paired with an XDP `REDIRECT`, DMA’s frames directly into a userspace memory region (a *UMEM* of fill/completion rings) with *zero copy* and no traversal of the kernel network stack. This is kernel-bypass packet I/O reachable from an ordinary process — the mechanism behind userspace networking frameworks — and it is the natural substrate for a specialized, connectionless bulk transfer of results or checkpoint images between nodes at line rate. The cost is that you inherit everything the stack was doing for you: `AF_XDP` delivers *frames*, not a reliable ordered byte stream, so segmentation, acknowledgement, retransmission, and congestion control are yours to build — which is precisely the RPC-from-first-principles exercise of Chapter 8, now at the packet level. Combined with the verifier’s limits on program size and complexity and the need for driver support, this makes XDP/`AF_XDP` a tool for a small number of hot paths — a load-balancing front end, or a bulk data mover — not a general RPC layer.

4.1.1 eBPF, maps, and the verifier

An XDP program is an *eBPF* program: restricted C compiled by `clang` to a sandboxed bytecode, loaded through the `bpf()` syscall, checked by the in-kernel *verifier*, and JIT-compiled to native code. The verifier is what makes running code in the datapath safe, and it shapes how these programs are written: it proves the program halts (loops must be bounded) and that every memory access is in range, so a frame is parsed by a strict *check-then-read* walk — a header pointer is compared against the frame’s `data_end` *before* it is dereferenced — under a small stack and a bounded instruction budget. Persistent and shared state lives only in *maps*: typed key/value tables the program shares with userspace (through `bpf()`) and across CPUs. Four map types carry this project’s datapath: a `BPF_MAP_TYPE_HASH` routing map (`function_id` → cell),

an `ARRAY` of per-function counters, and the two redirect targets `CPUMAP` (a pinned core) and `XSKMAP` (an `AF_XDP` socket). The program reaches the kernel only through *helper functions* — `bpf_map_lookup_elem`, `bpf_redirect_map`, and so on — some of which, redirection among them, require the program to declare a GPL-compatible license. If the packet comes from a blocked address, the program can immediately return `XDP_DROP`, preventing the packet from travelling further through the networking stack. If it is legitimate, it can return `XDP_PASS` and allow Linux to continue processing it.

This early execution point is what gives XDP much of its performance advantage: the closer packet processing is moved to the network driver, the less work the system needs to perform before making a decision about the packet.

4.1.2 Attachment modes and the reality of zero copy

The same program can attach in three ways, trading portability for cost. In *offloaded* mode it runs on a capable SmartNIC; in *native* (driver) mode it runs inside the driver’s receive poll, before any `sk_buff` exists — the intended mode, and the only one in which the NIC can DMA a frame straight into userspace memory; in *generic* (SKB) mode it runs later, in the stack after the `sk_buff` is built, which works on any interface (including the `veth` pairs used inside containers) but copies. The distinction is decisive for the zero-copy claim: true zero copy requires native mode *and* an `XDP_ZEROCOPY` socket binding on a driver that supports it, whereas the portable fallback — `XDP_COPY` in generic mode — exercises exactly the same routing logic but pays one copy. Where the benefit is real is therefore a property of the driver and mode, not of the program.

4.1.3 AF_XDP: the UMEM and its four rings

`AF_XDP` couples the `XDP_REDIRECT` verdict to a userspace socket through a shared memory region, the *UMEM*: a block of process memory, registered with the kernel and carved into fixed-size frames, that in native mode is the NIC’s DMA target so packets land in it with no copy. Four single-producer/single-consumer descriptor rings coordinate ownership. Two belong to the UMEM: **FILL** (userspace hands the kernel empty frames to receive into) and **COMPLETION** (the kernel returns frames whose transmission has finished). Two belong to the socket: **RX** (the kernel hands userspace descriptors of filled frames) and **TX** (userspace hands the kernel frames to send). A frame’s life is thus a cycle `FILL` → (DMA) → `RX` → processing → `FILL`. The bridge from the in-kernel verdict to one specific socket is the `XSKMAP`: userspace registers each socket’s file descriptor at its receive-queue index, and `bpf_redirect_map(&xsks_map, queue_id, ...)` in the program lands the frame in that socket’s `RX` ring. Because the rings are per queue, the design scales by binding *one socket per NIC receive queue* (spread by the hardware’s receive-side scaling) and pinning each to a core, which removes any shared reader and maps one-to-one onto a per-core worker pool; an optional `need_wakeup` flag lets the kernel signal when a `poll()` is actually required, amortizing syscalls at high rates.

4.1.4 How this project uses XDP

In our design XDP is the *routing plane* for the job data path, not a general transport. Each record travels in a UDP frame with a small fixed header (magic, `function_id`, `feed_id`, `record_id`, length) placed before the struct payload. An XDP program on the receiving node parses *only that header* — never the struct — reads `function_id`, looks it up in an eBPF *routing map* (`function_id` → target cell), and `XDP_REDIRECTs` the frame, either to a `CPUMAP` (a pinned core) or, for zero copy, into the target cell’s `AF_XDP` socket, before the kernel builds an `sk_buff`. The cell’s worker reads frames from its `AF_XDP` `RX` ring, hands the payload to its thread pool, runs the `dlopen’d` function, and writes results back through the `TX` ring. Two properties make this the right tool here: routing is *format-agnostic* (the payload’s struct/FlatBuffer layout stays

owned by the function; XDP touches only the header), and *relocation is a single map write* — when a cell moves to another node, updating `function_id` → target makes the stream follow it. The prototype in `experiments/xdp/` implements the parse-and-classify half (per-function counters in an eBPF map, driven by a UDP feeder); the production step replaces the counter with the `XDP_REDIRECT` into `AF_XDP`. Consistent with Chapter 8’s RPC-from-first-principles caveat, we use `XDP/AF_XDP` only on the hot, small-record path and keep ordinary sockets or `gRPC` for control and moderate streams.

4.1.5 Frame ownership across the pool, and in-kernel admission

Two consequences of the UMEM model matter once frames feed a thread pool rather than a single reader. First, ownership outlives the RX ring: a worker reads its job *in place* from the UMEM frame, so that frame cannot return to the FILL ring until the worker finishes; the reader records each frame’s address alongside the job and recycles it only after `run()` completes, and because several workers recycle into the single-producer FILL ring, those returns are serialized. If every frame is in flight the FILL ring drains and the NIC drops — a natural back-pressure bound set by the frame count and the queue depth. Second, the program can reject work *before* it costs anything: validating the header’s magic and destination in-kernel and returning `XDP_DROP` for non-conforming frames discards traffic aimed at the data-plane port before an `sk_buff`, a wakeup, or an allocation — the same mechanism that makes XDP a line-rate DDoS filter, here serving as admission control for the job stream, while conforming traffic on other ports is passed up untouched with `XDP_PASS`.

4.1.6 Deployment: privileges and containers

Because XDP is a kernel facility and containers share the host kernel, running the router or an `AF_XDP` receiver in a container is a question of privilege and namespace, not of a special runtime. Attaching the program needs `CAP_NET_ADMIN`; loading it and its maps needs `CAP_BPF` (or `CAP_SYS_ADMIN` on kernels before 5.8); the UMEM and maps are locked memory, so `RLIMIT_MEMLOCK` must be raised; and a runtime’s default seccomp policy typically blocks the `bpf()` syscall unless one of those capabilities is present. The program attaches to an interface *in the container’s network namespace*: on a bridge network that is a `veth` endpoint, which supports only generic mode and hence the copy fallback, whereas true zero copy requires a native-XDP NIC in the namespace — host networking, or an SR-IOV virtual function passed in, the arrangement the Kubernetes SR-IOV and `AF_XDP` device plugins provide. This is the same portability-versus-performance boundary as the attachment modes above, drawn now at the container edge.

4.2 gRPC: typed RPC on HTTP/2

`gRPC` is the opposite choice: a batteries-included, application-layer RPC framework and the de facto standard for service-to-service calls on Kubernetes. Interfaces and messages are declared in *Protocol Buffers*, whose compiler generates client and server stubs and whose tag-based binary encoding gives compact wire size and the backward/forward schema evolution of Chapter 8 (add optional fields, never reuse a tag). The transport is `HTTP/2`, which supplies — off the shelf — exactly the mechanisms that chapter built by hand: *multiplexed streams* over one connection, per-stream *flow control*, and header compression. Four call shapes fall out of it: unary, server-streaming, client-streaming, and *bidirectional streaming* — the last being the ideal form for the long-lived worker channel on which the coordinator dispatches tasks and the worker streams heartbeats and completions.

Around this core `gRPC` provides the production concerns as first-class features: *deadlines* propagated in call metadata (so a client’s deadline bounds all downstream work), cancellation,

status codes carrying retryability, pluggable *interceptors* for authentication, tracing, and metrics, and name resolution with client-side load balancing. In a cluster it slots directly into the Chapter 3 machinery: it traverses Ingress and Gateway controllers, gets mutual-TLS identity and retries from a service mesh for free, and has a standard health-checking protocol probes can use. For this project it is the natural control-plane transport — client→gateway→coordinator, and the coordinator↔worker channel — giving multiplexing, deadlines, streaming, and schema evolution without hand-rolling them. Its costs place it on the control side of the split: HTTP/2 framing and HPACK cost CPU at very high message rates, and because streams share one TCP connection a lost segment head-of-line-blocks all of them (a limitation HTTP/3 over QUIC removes), so gRPC is the wrong tool for the XDP-class bulk data plane.

4.3 XDP, io_uring and shared-memory IPC, together

Three mechanisms in this thesis turn out to be the same idea applied at three different boundaries: a ring buffer in memory that both sides map, with a producer and a consumer that advance their own counters and rarely need to ask the kernel for anything.

- **AF_XDP** (Section 4.1.3) at the boundary between the network card and a process: the UMEM is shared between the kernel and the process, and frames move through its fill, completion, RX and TX rings.
- **Shared-memory IPC** (Chapter 2, Section 9.2) at the boundary between processes on one machine: the arena of `engineJavaCore` and the link rings of `engineLinked`, where a message is one copy into pages the other process maps and a busy link makes no system call.
- **io_uring** at the boundary between a process and the kernel for everything else — files, sockets, timers. A submission queue and a completion queue are mapped into the process; it writes requests into one and reads results from the other, and a single `io_uring_enter` can submit a whole batch and wait for completions at once. With `IORING_SETUP_SQPOLL` a kernel thread polls the submission queue, so submitting needs no system call at all — the same trade as AF_XDP’s busy polling: a core spent spinning in exchange for no transitions.

Put together, they give the engine’s data path a shape in which no stage waits on the kernel more than it must, and payloads are copied as few times as possible.

```
NIC --> XDP program -----
| header valid, functionId in route_map? |
| no -> XDP_DROP (admission, before an sk_buff exists) |
| job -> XDP_REDIRECT via XSKMAP into ingress's AF_XDP socket |
| else -> XDP_PASS (TCP to the heads, etcd: the normal stack) |
v
ingress reads RX descriptors; payload from the UMEM frame, copied once
        into the shared store segment; frame returned to the fill ring
        | link ring + doorbell (shared memory, no syscall when busy)
        v
scheduler --> executors x N   input read in place from the store
        | link ring
        v
collector io_uring: WRITE <id>.bin.tmp, RENAMEAT to <id>.bin, SEND the
        result or an answer to a head -- one io_uring_enter per batch
```

The edge: XDP decides what is work. The program on the NIC already parses the application header without touching the payload (Section 4.1.3). Extended from counting to routing, it does three things: it drops frames whose magic or function identifier is not recognised

(XDP_DROP), so malformed or unwanted traffic costs nothing further; it redirects job frames through an XSKMAP to the AF_XDP socket of the node’s ingress process; and it passes everything else — the heads’ TCP connections, etcd — to the ordinary stack with XDP_PASS. The control plane and the data plane separate at the first instruction that sees the packet.

Inside the machine: shared memory carries the job. The ingress process takes each frame from the RX ring and copies its payload once into the shared store segment that every component maps, then returns the frame to the fill ring. That single copy is deliberate: it frees the frame at once, so a slow job cannot starve the NIC of frames (the back-pressure problem of Section 4.1.3’s frame ownership), and from then on the payload is never copied again — the scheduler passes a reference, and the executors read it in place. Placing the UMEM itself inside the store segment would remove even that copy, at the price of tying every frame’s lifetime to its job’s.

Everything else: io_uring. What remains are the operations that do go through the kernel: writing result files, renaming them into place, and talking TCP to the heads. Each is an io_uring request — WRITE, RENAMEAT, SEND, multishot ACCEPT and RECV with provided buffers for the head connections — so that a collector finishing a burst of results submits them all with one system call, and a receiver serves many connections from one completion queue instead of one blocking thread each. The same queue can wait on the AF_XDP socket’s readiness with POLL_ADD, which puts the network, the files and the heads behind a single wait point.

Where the prototypes stand. None of this is built yet. The XDP prototypes, xdpSoBuffer and xdpStartBuffer, implement the first half of the edge: a UDP datagram with a twelve-byte header (magic, function identifier, payload length), an XDP program attached in generic mode that parses it, looks the function up in a route map, counts it, and returns XDP_PASS, and a userspace server that receives it with recvfrom. They differ in how the job runs — one dlopes the function, the other is a freestanding worker with no C library that issues its four system calls directly — and the second shows how few kernel services a worker needs: exactly the ones io_uring would batch. The shared-memory rings exist in engineLinked. What joins them — the redirect into AF_XDP, the ingress that fills the store from the UMEM, and the io_uring loop around the collector — is the design above.

Costs. Each mechanism removes system calls by adding something to manage: UMEM frames to recycle, rings whose producers and consumers must never overrun each other, a polling thread or a busy core. All three need privileges or permissive policies in a container: bpf() and CAP_NET_ADMIN for XDP, locked memory for UMEM, and io_uring’s system calls, which some runtimes’ default seccomp profiles now block and which have to be allowed explicitly. And all three are Linux-only. The design is worth that only on the hot path — the stream of small jobs — and the rest of the engine keeps its ordinary sockets and files.

4.4 Where each fits

	XDP / AF_XDP	gRPC
plane	data (bulk, hot path)	control (API, dispatch)
layer	L2-L4, pre-stack / kernel-bypass	L7 on HTTP/2
unit	raw frames	typed messages + streams
reliability	you build it	provided (over TCP)
evolution	n/a (custom)	protobuf schema, tag-based
k8s fit	CNI dataplanes, L4 LB, offload	Ingress, mesh, mTLS, health
use here	result/checkpoint bulk transfer,	Submit/status, worker channel,

in-cluster load balancing

inter-component RPC

The two are complementary, not competing: they are the concrete realizations of the same control/data-plane split. gRPC carries the small, typed, reliable control traffic that must be correct and evolvable; XDP/AF_XDP (or, more modestly, plain TCP plus the object store) carries the large, latency-sensitive data traffic that must be fast. Choosing per plane, rather than forcing one transport everywhere, is the same discipline — match the mechanism to the guarantee it must provide — that governs every layer of this system.

Chapter 5

Protocol Buffers

The gRPC control plane of Chapter 4 rests on **Protocol Buffers** (protobuf) [8]: Google’s interface-definition language and binary wire format for structured data. It is the concrete realization of the schema-compatibility discipline this thesis returns to repeatedly — both ends must agree on a layout, and disagreement must be caught rather than tolerated. This chapter explains protobuf as a technology and then, in Chapter 7, exactly where it does and does not appear in this project — a distinction that matters, because the data plane deliberately uses a different, zero-copy cousin of protobuf rather than protobuf itself.

5.1 The schema and code generation

A protobuf schema is a `.proto` file: a set of *messages*, each a list of named, typed fields, and — decisively — each field carries an integer *field number* (its tag). The schema is the single source of truth; the compiler `protoc` generates, for each target language, the type together with its accessors and its (de)serialization code, so every participant speaks the same format without hand-written parsing. In `proto3` there are no required fields (all are optional with well-defined defaults), and the type system covers scalars (integers of several widths and signednesses, `bool`, `double`, `float`, `string`, `bytes`), `enums`, nested messages, `repeated` fields (lists), `maps`, and `oneof` (a tagged union). The field *name* is a source-level convenience; on the wire only the *number* identifies a field, which is the root of protobuf’s compactness and its evolution story.

5.2 The wire format

An encoded message is a flat concatenation of fields, each a key followed by a value. The key is a single varint packing the field number and a 3-bit *wire type*, `key = (field_number << 3) | wire_type`, and the wire type tells a reader how to consume the value even for a field it has never heard of:

wire type 0	varint	int32/64, uint32/64, bool, enum (sint via ZigZag)
wire type 1	64-bit	fixed64, sfixed64, double
wire type 2	length-delim	string, bytes, embedded messages, packed repeated
wire type 5	32-bit	fixed32, sfixed32, float

Varints store an integer seven bits per byte, the high bit marking continuation, so small numbers cost one byte; signed values use ZigZag (`sint32/64`) so that `-1, 1, -2, 2...` map to `1, 2, 3, 4...` instead of producing ten-byte negatives. A length-delimited field is a varint length then that many bytes, which is how strings, opaque `bytes`, and nested messages nest uniformly. Concretely, a message `{ function_id = 7; payload = AA BB CC }` encodes as:

```

field 1 (function_id=7): 08 07
    08 = (1<<3)|0 (field 1, varint)    07 = value 7
field 4 (payload, 3 bytes): 22 03 AA BB CC
    22 = (4<<3)|2 (field 4, length-delim) 03 = length  AA BB CC = bytes

```

Two properties follow directly. Fields are *order-independent and self-delimiting*: a reader keys off the field number and skips unknown fields using the wire type, so the message is not positional. And field *names never travel* — the wire carries numbers and values only — which keeps it compact but means the `.proto` is mandatory to interpret the bytes. This is the exact opposite of the data plane’s flat buffer, where fields live at *fixed offsets* with no tags and are read in place with no decode (Section 7.3).

5.3 Schema evolution

Because a field is identified by number and unknown fields are skippable, a schema can change without breaking peers built against an older or newer version, provided a few rules hold: a field number, once used, is never reused or repurposed; new fields take new numbers and, being optional, default cleanly when absent; and retired numbers/names are **reserved** to prevent accidental reuse. Old readers ignore (or, in many implementations, preserve) fields they do not know; new readers see defaults for fields an old writer omitted. This is precisely the “add optional fields, never reuse a tag” backward/forward compatibility that Chapter 4 credited to the control plane, and it is what lets a running cluster be upgraded component by component rather than in lockstep.

5.4 Implementing the wire format

The previous sections describe the format. This one reports what implementing it teaches. The codec behind this project’s control plane is written by hand — `cluster/src/pb.c`, about two hundred lines, no dependencies — rather than generated by `protoc`. In any other setting that would be the wrong choice: generated code is correct, complete and free.

What generation hides is that the properties attributed to Protocol Buffers in §5.3 follow from two decisions visible in twenty lines of a decoder.

5.4.1 Forward compatibility is one line

Every decoder in `msg.c` has the same shape:

```

while (pb_read_tag(&r, &field, &wire)) {
    if (!wire_legal(wire)) return -1;
    switch (field) {
        case 1: m->function_id = (uint32_t)pb_read_varint(&r); break;
        ...
        default: if (pb_skip(&r, wire) < 0) return -1;          break;
    }
}

```

The `default` arm is schema evolution. Not a library feature, a negotiation or a version handshake: a `switch` default that steps over what it does not recognise. A worker compiled against last month’s `.proto` decodes this month’s coordinator correctly, ignoring what it cannot name. It can step over those fields because the wire type travels in the tag, so the length of a value is computable without knowing what the value means. “Add fields freely, never reuse a tag number” is not a convention enforced by tooling; it is the condition under which that one line is sound.

The flat buffer’s counterpart is the absence of the loop. No tags, so nothing to skip, so no evolution. Two hundred lines of decoder against zero is the trade.

5.4.2 Default elision, and what it costs

A proto3 scalar equal to its default never reaches the wire, so every field writer starts with a test:

```
void pb_put_uint32(pb_writer *w, uint32_t field, uint32_t v)
{
    if (!v) return;                /* absent == zero, on purpose */
    pb_put_tag(w, field, PB_VARINT);
    pb_put_varint(w, v);
}
```

Hence a message whose fields all hold defaults encodes to zero bytes, a case the conformance vectors check. There is a real loss here: for a proto3 scalar, “absent” and “present and zero” are the same bytes and no reader can tell them apart. Proto2 had explicit presence, proto3 removed it, and later versions restored it only for `optional` and message-typed fields.

This shapes schema design directly, and the project’s schema shows it. Status codes put zero on success and every failure on a non-zero value, because a status using zero for a failure mode would be indistinguishable from an absent status. That is forced by the encoding, and it is the sort of coupling between wire format and schema that a generated codec makes easy to miss.

5.4.3 The fiddly parts

Three details account for most of the difficulty, and a naive version of each is silently wrong rather than obviously broken.

Submessage lengths precede bodies of unknown length. A length-delimited field is a varint length then that many bytes, so encoding a nested message needs its size before writing it. The usual answers are to encode twice, or to encode into scratch space and prepend. This implementation reserves a two-byte placeholder, writes the body, computes the true varint width and `memmoves` if they differ. Correct at any size, with one small move in the uncommon case. Two bytes covers bodies up to 16,383, which is every control-plane message here, so the move never happens in practice — and the vectors include a 200-byte submessage to exercise the branch that would otherwise go untested.

Varints have a maximum length, and exceeding it is undefined behaviour rather than merely wrong. A 64-bit value needs at most ten bytes ($\lceil 64/7 \rceil$). A decoder still accumulating on an eleventh shifts by seventy bits, and a shift count at least as wide as the promoted type is undefined in C.¹ So malformed input is a language-level problem, not a data problem, and the check that rejects it is required for correctness rather than defensive.

Every length off the wire is attacker-controlled. The reader bounds each one before use, and writes each comparison as a subtraction, `1 > r->len - r->pos` rather than `r->pos + 1 > r->len`. The addition can wrap and a wrapped comparison passes.² The same rule governs flat-buffer offset validation on the data-plane side, which is the more dangerous of the two: a flat buffer is dereferenced, not copied.

¹ISO/IEC 9899:2011 §6.5.7p3.

²CERT INT30-C.

5.4.4 Checking the claim

An implementation that merely resembled Protocol Buffers would pass any test written against itself. So the claim is checked against an independent encoder: `cluster/test/gen_vectors.py` implements the same specification, in another language, without reference to the C, and `make check` compares the two byte for byte across nine messages. Two implementations derived independently from a specification agreeing tells you something about the specification. An implementation agreeing with its own test tells you it is deterministic.

The vectors include the awkward cases on purpose: an all-default message at zero bytes, a maximal ten-byte varint, negative `int64` values widened to full two's complement — the regime `sint64` and ZigZag exist for, and where choosing `int64` costs ten bytes a field — the two-byte submessage prefix, and four malformed shapes that must be rejected rather than survived. A last test appends fields with numbers the build has never seen and confirms the known ones still decode, reducing §5.3 to an assertion.

5.5 gRPC and Protocol Buffers together

The same `.proto` that defines the messages also defines the *service*: a set of `rpc` methods, each naming a request and response message and optionally marking either as a `stream`. `protoc` with the gRPC plugin generates the client stub and server skeleton, and gRPC carries length-prefixed protobuf messages over HTTP/2, supplying the multiplexing, flow control, deadlines, and status codes that Chapter 4 described. The four call shapes (unary, server-, client-, and bidirectional-streaming) are just which sides are marked `stream`; the wire encoding of each message is the format above.

Chapter 6

Coordination: Algorithms and the Services That Implement Them

The engine of Chapter 7 will avoid coordination by construction, and the distributed engine of Chapter 8 will buy the little it needs from `etcd` rather than implement it. Neither decision is defensible without some account of what the alternatives actually promise. This chapter is therefore short and deliberately narrow: the problem that makes agreement hard, the algorithm that solves it, the services that package that algorithm, and the cheaper protocols that are correct when agreement is not what is wanted. Each is stated with the reason it does or does not appear in this project.

6.1 Why agreement is hard

Three results bound everything that follows.

Failure detection is impossible to do perfectly. Over an asynchronous network a process that has not answered is indistinguishable from a process that is slow, and no timeout separates the two. Every practical system therefore picks a timeout and accepts that it will sometimes be wrong, which means the interesting question is never “how do we detect failure” but “what does the system do when we detect it wrongly”.

FLP. In an asynchronous model with even one faulty process, no deterministic protocol can guarantee that all correct processes agree in bounded time [12]. Real systems escape this not by cleverness but by weakening the model: they assume partial synchrony — that the network is eventually well behaved for long enough — and then guarantee safety always and liveness only during those periods.

CAP. Under a network partition a system chooses between remaining consistent and remaining available [13]. The choice is not global: it is made per piece of state. This project makes it twice and differently — cluster membership is consistent and may become unavailable, while the data path is available and never needs agreement at all, because content addressing makes a repeated answer the same answer.

6.2 Consensus: Paxos, and then Raft

Consensus is the problem of getting a set of processes to agree on one value, or equivalently, on one ordered log of values. Paxos [14] is the original solution and is correct, but it is famously difficult to derive a working system from, because it specifies agreement on a single value and leaves the composition into a replicated log to the implementer.

Raft [9] solves the same problem and was designed for understandability, by decomposing it into three pieces that can be reasoned about separately.

- **Leader election.** Time is divided into terms. A follower that hears nothing from a leader becomes a candidate, increments the term and asks for votes; a majority elects it. Randomised timeouts make split votes rare and self-correcting.
- **Log replication.** All writes go to the leader, which appends and replicates. An entry is *committed* once a majority has stored it, and only then applied. A majority of any two majorities intersect, which is the whole reason a committed entry cannot be lost.
- **Safety.** A candidate whose log is behind cannot win an election, so a new leader always holds every committed entry.

Two consequences matter for design rather than implementation. A quorum is $\lfloor N/2 \rfloor + 1$, so clusters are odd-sized and a five-member cluster tolerates two failures — adding members buys fault tolerance, never throughput, since every write still costs a round trip to a majority. And membership change is its own protocol (joint consensus), not a configuration edit — which is why a quorum’s size is a correctness parameter rather than a capacity knob, and why membership change, not failure, is the first thing that breaks the ownership argument of Section 7.6.

6.3 Leases, heartbeats, and membership

A *lease* is a lock with an expiry, and it is the mechanism by which a consensus store becomes useful for liveness rather than only for storage. A participant holds a lease and renews it; the store — not any peer — expires it when renewal stops. Because expiry is decided by the same replicated state machine that decides everything else, every observer agrees on when the holder went away, and no participant has to detect the failure or clean up after it.

The cost is that a lease is a bet on clocks: a holder that is merely partitioned loses its lease while still running, and must therefore stop acting on its authority when it cannot renew. This is exactly the behaviour Section 8.5 relies on — presence in the store is a live claim rather than a record — and it is also the reason a partitioned node’s data is not advertised even though the bytes still exist on its disk.

6.4 The services

etcd is Raft with a key-value API: linearizable reads, atomic compare-and-swap, leases attached to keys, and *watches* that stream changes to clients instead of making them poll. It is the backing store of Kubernetes (Chapter 3), which is the strongest available argument about how well exercised it is, and it is what this project uses.

ZooKeeper [15] predates it and solves the same problem with its own protocol, ZAB. Its data model is a tree of *znodes*, and two flavours give it most of its power: *ephemeral* nodes disappear when the session that created them ends — the same idea as a lease — and *sequential* nodes receive a monotonically increasing suffix. Those two, plus watches, compose into the standard recipes: a lock is the lowest sequential child, and leader election is a lock. It was not chosen here mainly for operational weight, being a second JVM with its own configuration and failure modes.

Consul combines both styles: Raft for the consistent catalogue, and a gossip layer for membership and failure detection. It adds service discovery and health checking, neither of which this project needs, since a node’s presence claim already is its health check.

6.5 Gossip, when agreement is not required

Not all shared state needs consensus. Load figures and holdings maps are advisory: acting on a value that is two seconds stale costs a suboptimal placement, not a wrong answer. For state of that kind an epidemic protocol is a better fit. In SWIM [10] each member periodically

probes a random peer, asks other members to probe on its behalf before declaring anything, and piggybacks membership updates on those probes. Failure detection is by suspicion, dissemination is eventual, and the load is spread across all participants with no leader and no quorum.

The trade is precisely stated: gossip scales far better and answers a weaker question. At two nodes it would be more code for less certainty, which is why it was not adopted here; at a larger node count it is the first thing this design should try, because the state it would carry never needed to be linearizable.

6.6 Brokers: Kafka and RabbitMQ

A queue between the nodes is the obvious way to distribute work, and it is worth being explicit about what the two dominant shapes offer.

Kafka [16] is a partitioned, replicated log. Producers append to a partition, consumers track their own offset, and ordering is guaranteed within a partition but not across them; replication follows a leader with an in-sync replica set, so durability is tunable by acknowledgement policy. It is excellent at replay and at fan-out to independent consumer groups.

RabbitMQ is a traditional message broker: queues with routing, per-message acknowledgement, redelivery on failure, and back-pressure through bounded queues — the last of which this project genuinely lacks.

Both were rejected for the same structural reason rather than on features. A broker is a store-and-forward intermediary: work is written to it and read from it, so every job — and, if payloads travelled with jobs, every byte — would traverse a third machine. The entire argument of Section 8.2 is that a 96 byte job should travel to where the megabytes already are. An intermediary in the middle makes that impossible by construction, and it is the same objection that rules out putting jobs into `etcd`.

6.7 Placement without a directory

Once results are content-addressed, locating them can become arithmetic rather than lookup. Hashing an identifier modulo the node count is the naive version and is what the implementation currently does; it is correct but reshuffles almost every key when a node joins or leaves. *Consistent hashing* [17] places nodes and keys on a ring so that a membership change moves only the keys adjacent to the change, and *rendezvous hashing* achieves the same $1/N$ movement by scoring every node for each key and taking the maximum. A full distributed hash table such as Chord [11] adds routing so that no node needs the full membership list — unnecessary at a scale where every node can hold every address, but the natural destination if this design grew.

6.8 What this project uses, and why

The design splits its state in two and treats each with the cheapest mechanism that is correct for it.

- **Off the data path**, membership and holdings are small, must be correct, and change rarely: they go into `etcd`, with leases doing the failure detection. Two keys per node per second is a workload consensus is entirely comfortable with.
- **On the data path**, jobs, code and results need no agreement at all. Identity is derived from content, functions are pure, and inputs are immutable, so a repeated dispatch is the same dispatch and a lost result can be recomputed. There are no locks, no transactions and no two-phase commit anywhere in the system — not because they were avoided as a matter of taste, but because content addressing removes the question they answer.

Two-phase commit deserves the explicit note, since it is the reflexive answer to “two machines must both do this”. It blocks: if the coordinator fails after the prepare phase, participants hold their locks until it returns, and the protocol has no non-blocking recovery. Idempotence over immutable, content-addressed inputs achieves what it is usually reached for, and does so without a coordinator to lose.

Chapter 7

The Engine

In the preceding chapters technologies were discussed, that are used today to enhance distributed systems in the production level. Kubernetes can automate scaling in cloud environments, acting like the operating system of the cloud, find its best application managing multiple cpus across various machines, but as is evident it's best applied when used on specific hardware, with localization, in which case communication costs drop by orders of magnitudes and provide a great setting for the distributed architectures in use today. What became also evident was that the communication costs usually far outweigh the benefits of having an extra cpu or multiple of them across the room or the city/world, benefits are mostly measured with latency. As a result, despite the higher development time it certainly pays to incorporate gRPC or a protocol buffer architecture in both microservices setups or more scientific/compute focused applications. Finally, the epitomy of speed on computation, is the XDP, here the eBPF suite is able to provide a seemingly magic solution, which is subject to a number of limitations however, and it's mostly used as a logging aggregation tool. The discussion on execution and syscalls in linux, brings us to the CRIU project, which is not best suited for relocation of isolated processes, and in a healthy cluster would not make sense to relocate a running container via that, it could however find applications on a similar project.

The engine is deliberately a *subset*. It implements job definition, identity, placement, dependency tracking, code shipping, and the feed of one job's output into another's input. It does not implement failure recovery, concurrent dispatch, or multi-head routing. Section 7.10 states each omission and where it would attach, because an honest account of what a prototype does not do is more useful than a diagram of what it might.

7.1 The high level design

The objective of this work is to design a system that's capable of utilizing two or more computers in order to perform a certain task or an arbitrary number of them faster than one of the machines would be, but not in an embarrassingly parallel or trivial way. What I have chosen to design is a somewhat loosely coupled system where communications are still prevelant and extremely important, but are baked in the system in a way that is scalable, and configurable. For example the kubernetes management of the lifecycle could provide the basis for dynamically adding nodes to the network, and spawning worker nodes there. Also since the aim is not to just fan out or map reduce, but achieve actual results based on non trivial dependencies of the jobs executed we opted for a design that lets one submit for computation a series of codependent jobs. Actually, codependent may is not the right word here, since what we define is a DAG: directional acyclic graph that defines dependencies of jobs, defining which jobs need the output of others to start processing. One or more jobs may be used as input to the "sink". Also with the use of thread pools on the workers we can actually implement rather efficiently a somewhat trivial fan out technique, that's configurable i.e by setting a header flag as parallelizable and in so doing being

able to break down the fed data into multiple subcopies.

7.2 The job model

The definition of the work "unit" in the system is the *job*, it starts its life as a pair of data and a function. The function is a loadable .so static object file that can be loaded dynamically, the .sos are going to be distributed along the working nodes and not retransmitted via the wire on runtime, since most systems can be designed with the expectations of the loads that they are going to face in mind. Also large jobs are going to be transmitted via TCP for payload protection, since a lost data piece would halt a whole pipeline and perhaps destroy resulting data making them unusable. What is being implemented is a naming scheme with a tag for a data set and each pre-distributed function, a pair of which constitutes a job and which is also assigned an id, these ids could be hashes so that they are easily calculated and hashes could easily be used to help implement result storage, say for example via a distributed hash table. The goal is to have a configurable yet easily adaptable to additions of exemptions of nodes for result storing or computation when needed, and most importantly for a distributed system, this would allow the system to function without having to distribute the information to each node: if the result is easily traceable then not everyone needs to know about all the parts of a graph, and where they are being executed, and if someone needs to read one they can easily get the job id, hash it and calculate the node on which it's been saved. Of course the worker could hold no catalogue of functions and it would be beneficial for it to not need no prior knowledge of the work it is about to do; everything it must know arrives in the bytes it has just read, a short lived job could no longer burden the system if it's been assigned a 'no cache' flag.

In most cases the unit of data is a *dataset*: rows of fixed-width integer vectors, each row carrying its own identifier. The unit of code is a compiled shared object, or more promptly, it's an id for the .so that's been assigned to it via a hash, so that it's consistent, and we can have it transmitted via a leader after start up of either the vms, the machines or the containers that are part of the 'cluster'. A *jobGraph* is a set of jobs together with their dependencies, and a job's input is either a file or the concatenated rows of its dependencies' outputs. The graph is defined as a set of jobs, and the jobs having as intake the output of another have as input the id of the job. The DAG is constructed and then reversed, this way we know which jobs need to take place first. After this has happened in general a loadbalancer could make the decision to either assign the job to the local machine or dispatch it towards the network. The loadbalancer could be L7 application via kubernetes, or L4 via kubernetes or L4 via the actual connections the heads are managing. Ofcourse, it could also be an issue of storing the result, one could argue that the result is best saved on an appropriately chosen machine, for example a hash of the job modN could give us the id of the worker node, that'll store the results, with the caveat that the network can be easily saturated. In fact, for a simple internet connection one could have up to a 1MB/s of upload, and jobGraphs should output a significant amount of data if they are to benefit from this model of execution, which in this DAG-approach is a synchronized (weakly) pipeline generator.

```
worker 127.0.0.1:16001
worker 127.0.0.1:16002
```

```
job fibA  so=funcs/fib.so fn=fib data=input/vectors_a.bin flags=parallelizable
job fibB  so=funcs/fib.so fn=fib data=input/vectors_b.bin flags=parallelizable
job total so=funcs/sum.so fn=sum deps=fibA,fibB
```

Two producers compute Fibonacci numbers over independent inputs; a third consumes both. That last line is the whole point of the exercise — it is the edge across which one job's result becomes another job's input, on a different machine, without either job knowing the other exists.

Two programs implement this. The **head** reads the graph, assigns identities, decides placement, tracks dependencies, and dispatches. The **worker** receives one task, loads the function it carries, runs it over the data it carries, and replies. There is no third component: no coordinator, no metadata service, no queue.

7.3 One wire format

The experimental groundwork for this engine was four client/server pairs, each exploring a different execution strategy, and each of which grew its own frame layout — five wire formats and three incompatible function signatures across six projects. The engine’s first design decision is therefore a negative one: there is exactly one definition of the format, and the head, the worker and every job function compile against it.

```
task frame      head -> worker
[u32 totalLength][ 48-byte header | symbol | data | code ]

    0 magic  0xE9C1FEED      20 nameOffset  24 nameLength
    4 version                                28 dataOffset  32 dataLength
    8 jobId  (u64)             36 codeOffset  40 codeLength
   16 flags                                44 reserved

reply frame     worker -> head
[u32 status][u32 outLength][outLength bytes]

dataset
[u32 magic][u32 version][u64 rowCount][u32 cols][u32 reserved]
rowCount x [u64 rowId][int64 v0 .. v(cols-1)]
```

Offsets rather than a packed struct: the frame is self-describing, so the regions can be validated independently and a stale sender fails on magic or version rather than by silently misreading a field. The reply carries a **status**, which is a small thing that matters — a failed job is *reported*, where the earlier prototypes signaled failure by dropping the connection and left the caller to infer the cause.

One class of bug in those prototypes is worth recording, because it appeared independently in every one of them. Each validated an attacker-controlled row count with arithmetic that overflowed before the check could reject it: a count near 2^{61} makes $8 + \text{count} \times 8$ wrap to a small value, the guard passes, a small buffer is allocated, and the loop then writes past it. In the datagram-based pairs this was reachable from a single unauthenticated packet. The engine’s `dsBytes` bounds the count *before* multiplying, which is the only ordering that is safe, and the constant that bounds it (`DS_MAX_ROWS`) is part of the format rather than a local convention.

7.4 Identity: ids computed from the bytes

Every identifier is a hash of the bytes it names: **the first 64 bits of their SHA-256 digest, read big-endian, so that an identifier printed in hexadecimal is exactly the first sixteen characters of `sha256sum`:**

- a **row id** is the hash of that row’s value bytes, so a row keeps its identity no matter which job produced it or how many merges it has survived;
- a **job id** is the hash of the function name, then the code, then the input, so the same function over the same input is the same job — on any head, in any process, forever.

SHA-256 replaced the FNV-1a hash the engine began with. FNV-1a is fast but not cryptographic: bytes that hash to any given identifier can be constructed directly, and in a content-addressed system that is not an inconvenience — it serves the wrong code or the wrong data

under a name that checks out. With a truncated SHA-256, forging bytes for an existing identifier costs in the order of 2^{64} hash evaluations. Keeping 64 bits leaves every identifier field, header and log line unchanged; the price is that two colliding objects could be manufactured in about 2^{32} evaluations, and that accidental collisions become likely only as the number of distinct objects approaches 2^{32} .

This is content addressing, and it is cheap to adopt at this scale while paying for itself several times over. It gives deduplication for free: two submissions of identical work collide by construction. It gives idempotence, because a repeated dispatch is recognizably the same dispatch. It gives a cache key for the function artifact, which is the mechanism by which the shared object stops traveling with every task. And it gives determinism that can be checked from outside the system: the row identifiers produced by the engine were reproduced by an independent implementation written against the format specification alone, which is a stronger statement about the format than any amount of internal testing.

This is something that could be implemented either via a redis store, or a postgres store procedure (especially since we know of the ways that the results or general content payloads are going to be in the network).

Inputs are content-addressed and job functions are pure, so any lost result can be recomputed by replaying its subgraph. The ingredients for lineage-based recovery — the model Spark uses in place of replicating intermediate data — are already present; only the bookkeeping is missing, but if the 'leader' or outwards facing head node can store the This is noted again in Section 7.10.

7.5 Placement by reverse traversal

The head places jobs by walking the graph *backwards*. Sinks — jobs nothing depends on — are assigned first; each producer is then placed relative to the consumer that was placed before it.

The ordering is what makes this worth doing. Once a consumer has a node, every producer above it knows where its output must go *before it runs*. Results can therefore be pushed forward to a destination that is already known, rather than published somewhere and then discovered by a lookup. A design that places producers first has no such luxury: it must either buffer results centrally or introduce a directory service to answer the question that reverse traversal answers for free. This is describing however only one possible implementation to solve the issue. The main problem is that a zig zag across machines would be stupidly expensive if it comes with carrying along data that are large enough to make parallel execution worthwhile i.e. huge data sets and complex functions, since as we have seen many times over the communication costs are prohibited especially when talking about internet connections, and initializing of connections and data transfer, so unless the workload is extremely large, a 'local' - greedy balancer would make things perhaps worse if left unchecked.

A strong bias for cpu affinity would come into play especially if all head nodes were to receive job sets, resulting to dispatching in small cases and in so doing basically parallelizing most of the stuff.

For a start howeve one could try this free standing policy: a producer inherits its consumer's node by default, which keeps the feed node-local; a producer marked **parallelizable** in the graph is spread to the next node instead. With the three-job graph above:

workers	fibA	fibB	total	
1	w1	w1	w1	
2	w2	w1	w1	
3	w2	w3	w1	
4	w2	w3	w1	(three jobs; w4 idle)

The flag is the only scheduling hint the graph carries, and it is enough to express the distinction that matters here: work that must be near its consumer, and work that should be spread away

from it. The main problem solved is a lack of need of communication in cases where results are ready in one or two nodes and one other node needs to find them and collect them to start processing, something that could however be solved by knowing the jobs hash and sending the bytes that way, with the speculation that the job is there, is ready to run, and that it won't bottleneck execution on multiple fronts or is not going to deadlock multiple -not threads- but machines.

7.6 Dependency tracking without consensus

Each job holds a counter of unsatisfied dependencies. When a job completes, the counter of every job depending on it is decremented; a job whose counter reaches zero is ready to dispatch. This is the standard dependency-counter formulation, and the standard treatment of it spends considerable effort on the race: two producers finishing simultaneously, both reading the same counter, both writing the decremented value, and the join silently never firing. The usual remedies are an atomic decrement in a transactional store, or a coordination service holding the counter on everyone's behalf.

The engine needs neither, and the reason is structural rather than clever. Placement is deterministic and the graph is known, so each job has exactly one owning head, and only that head ever writes that job's counter. A single writer cannot race with itself. The concurrency problem was not solved; it was *removed*, by an ownership rule that placement already had to establish for other reasons.

This property is worth stating precisely because it is the part that does not survive carelessly. It holds as long as ownership is unambiguous — which means it depends on every head computing the same placement, which in turn depends on every head seeing the same membership. Membership change, not failure, is the first thing that will break it, and it is the reason the design calls for a membership epoch stamped onto each graph at submission.

7.7 Shipping code

The worker receives the function as bytes and must run it without the artifact ever existing as a file. It creates an anonymous in-memory file with `memfd_create`, writes the received object into it, and calls `dlopen` on `/proc/self/fd/N`. Nothing touches disk, and because each task gets a fresh inode, the dynamic linker's path-based cache cannot return a previously loaded object that happened to share a name — a failure mode that is invisible until two jobs disagree about what a symbol means.

This mechanism was developed in the first of the four prototype pairs and is reused unchanged. The prototypes established a ranking between the three ways a worker can obtain code, fastest first: a resident function compiled into the worker; flat position-independent machine code copied into a mapping made executable with `mprotect`; and a shared object loaded through the dynamic linker. The engine uses the slowest of the three deliberately. The ranking measures the *loading* step, which content addressing is about to make rare, and in exchange the shared object brings a symbol table, a defined ABI, and ordinary tooling — a flat blob has none of these, and its ABI in the freestanding prototype omitted the output capacity argument, which is precisely how it came to write past its buffer.

7.8 Two heads: C and Java

The head was built twice, in C and in Java, over an identical worker, an identical format, and identical job functions. The only variable is the head's language; the build refuses to proceed if the shared files have drifted, since at that point the comparison means nothing.

The port is not free, and the cost is exactly one file. In the C engine the format has a single definition that all three parties compile against, so drift is impossible by construction. A Java head cannot participate in that: it needs its own transcription of the frame layout, maintained by hand, with nothing in either language able to detect the moment the two stop agreeing. Three specific hazards appear inside that transcription, and all three produce plausible-looking wrong answers rather than errors: Java's `ByteBuffer` defaults to big-endian where the C side writes native little-endian; `DataStream.readInt` is likewise big-endian and cannot be used to read the reply header; and **the identifier must be read out of the SHA-256 digest big-endian — the one place where the Java default is right, and where reading it little-endian like every other field would reverse each identifier relative to the C implementation's.**

What the port buys is equally concrete. The dataset, a family of loose functions over a `void *` in C, becomes an object that knows its own shape, and merging several datasets becomes four lines. Graph parsing and the placement walk are similarly shorter and harder to get wrong.

The conclusion matches the split this thesis has maintained throughout, and the line falls where the bytes are. Java is the better language for the graph, the placement policy, the parsing, and — the argument that carries the most weight in a Kubernetes deployment — for watching the API server, which is substantial work in C and nearly free with a client library. C is the better language wherever payload is touched, and it remains the only reasonable choice for the worker. The productive arrangement is therefore not to choose, but to split the head: control in Java, byte movement in C, with the format defined once on the C side. A head that stays off the data path can be written in either language; a head that carries results cannot be written in Java without paying for it.

7.9 Deployment

The engine runs as a set of containers, one per node, on a user-defined network so that nodes address each other by name rather than by address. A single image serves every role — build environment, worker, and head — and the project directory is bind-mounted rather than copied in, so that rebuilding a binary does not mean rebuilding an image. Workers therefore all listen on the same port, distinguished by hostname.

The nodes are provisioned as debugging environments rather than as minimal runtime images: each carries `gdb`, `strace`, `tcpdump` and the traffic-control tools, and runs with the capabilities those require. This makes the properties discussed above directly observable — the frame header can be read off the wire between two containers, the `memfd_create` and `dlopen` sequence can be traced as a task arrives, and artificial latency or packet loss can be introduced on a single node's interface to see how the engine behaves when the network stops being ideal.

Kubernetes is intentionally absent from this iteration. Everything Chapter 3 describes applies — a worker fleet is a Deployment, the head is a StatefulSet with stable per-pod DNS, and membership comes from the API server — but the engine's own mechanisms are easier to observe and to falsify when the orchestrator is not also in the picture. The step from named containers on a network to named pods on a cluster is small, and it is the natural next one.

7.10 What the engine demonstrates, and what it does not

Running the three-job graph across two worker containers produces the expected placement, the expected seven result rows, and identical output from the C head and the Java head. Two invariants are worth separating, because they fail differently. Row identifiers depend only on row values and must match across heads, compilers and machines. Job identifiers additionally hash the function's compiled bytes, so they are stable only for a fixed toolchain — rebuilding the functions with a different compiler moves every job id while leaving every row id untouched.

This is the correct behaviour for a content-addressed system and an easy result to misread as a bug.

The omissions are as follows.

- **Failure handling.** A worker that cannot be reached fails the whole graph. There is no retry, no re-placement, no timeout. As Section 7.4 notes, content-addressed inputs over pure functions already permit recovery by recomputation; what is missing is the bookkeeping to know what to recompute, not the ability to do it.
- **Concurrent dispatch.** Independent jobs placed on different nodes are dispatched one after another. Placement is therefore demonstrated but not yet exploited, and no speedup should be claimed from the current implementation.
- **A single head.** One head owns every job in the graph, which is what makes the single-writer argument of Section 7.6 trivially true. The claim becomes interesting, and requires the membership epoch, only with several heads.
- **Function caching.** The shared object is re-sent with every task, although the job id already contains the hash that would make a per-node cache correct. This is the shortest path from the current prototype to a meaningful reduction in bytes on the wire.
- **Transport selection.** One transport is used for everything. The size-based split between a bulk path and a low-latency path, and the XDP work of Chapter 4, remain design rather than implementation.

What the engine does establish is the part that is hardest to retrofit: a single authoritative wire format, identity derived from content rather than assigned by a counter, a placement rule that makes destinations known before results exist, and an ownership rule that eliminates the coordination problem instead of solving it. Each of the omissions above attaches to that skeleton without disturbing it, which was the objective.

Chapter 8

The Distributed Engine: Architecture and Alternatives

Chapter 7 described an engine with a single head, a task frame carrying its own payload, and a set of workers reached over one transport. That prototype established the parts that are hardest to retrofit — one authoritative wire format, identity derived from content, placement decided before results exist — and it named its own omissions in Section 7.10.

This chapter describes what the system became once those omissions were attacked. The wire format was rewritten around self-describing objects rather than framed tasks; the worker grew a store, a queue and a work-stealing pool; the head was reduced to a submitter and split into three entry points; and a coordination layer was added so that several machines can find each other and move work between themselves. The chapter closes with the technologies that were considered and not used, and the argument for each decision — Raft in particular, since the system depends on it without implementing it.

8.1 What changed, and why

The earlier design sent a *task frame*: a header, the symbol to call, the data, and the compiled function, all in one message, with a length prefix in front of it. It worked, and two properties of it did not survive contact with the rest of the problem.

The first is that a frame carrying its payload cannot be cheap. Every dispatch moved the data and, in the first version, the shared object as well — the benchmark chapter noted 512 jobs each carrying a fresh 15.5 KB copy of the same function, roughly 8 MB of redundant transfer for one run. The fix is not a better frame; it is to stop putting the payload in the message at all.

The second is that a frame is not addressable. Once the head no longer waits for a reply, and once more than one machine is involved, the interesting operations are *name* operations: does this node hold that dataset, where did that result go, send me the object with this identifier. A format whose unit is “one task, with everything in it” has nowhere to put those questions.

The rewrite therefore replaced the frame with four *objects*, each of which is a thing that can be named, stored, requested and transferred independently. What a job is did not change. What a job *carries* did.

8.2 The object model

Every object begins with the same twenty-four byte envelope:

offset	size	field
0	4	magic
4	2	version
6	2	type
8	8	objectId
16	8	totalLength

and there are four types.

- **DATA** — 56 byte header plus an opaque payload. Bytes with an identity. A *result* is a DATA and nothing else, which is what allows one job’s output to be another job’s input with no conversion anywhere.
- **FUNCTION** — 112 byte header plus the compiled shared object. The header carries the ABI version and a fixed 64 byte symbol field, so the code always begins at a constant offset.
- **JOB** — 96 bytes, fixed, carrying nothing. The function, the input and the output are named by identifier. Also the parent job, the part index and count for a divided job, the state, and the source and destination nodes.
- **REQUEST** — 56 bytes, fixed. “Send me the object of this type with this identifier.”

In the current implementation. Three fields have changed meaning. The FUNCTION’s 64 byte symbol field is still in the header, so every offset is unchanged, but nothing reads or writes it: a function has no name other than its identifier. Every job `.so` exports one fixed entry point, `engineJob`, declared once in `contract.h` as `JOB_FUNCTION_ENTRY`, and the worker looks up nothing else. The four bytes that closed the JOB header as reserved are now `JOB_FLAGS`, whose one flag, `DEPENDENT`, marks a job whose input is another job’s result. The part fields — parent, part index and part count — remain on the wire but are unused, because the division of jobs into parts was removed (see Section 8.6).

Three consequences follow from this layout, and they are the reason for it.

Framing is free. `totalLength` sits at offset 16, inside the first twenty-four bytes. A reader takes the envelope, learns the length, reads the remainder, and the next object begins immediately after. There are no delimiters, no escaping, and therefore no byte sequence that a payload must not contain. The same bytes are valid on a socket, in a pipe, and in a file, so a job written to a file today is byte-identical to the job that would have arrived over a connection — which is what lets a single reader implementation serve all three.

A job is 96 bytes. Moving work between machines costs less than a network packet. Moving its *data* may cost megabytes, and the two decisions are now separable. This is the single most important property for a system whose links are ordinary Ethernet: the correct instinct becomes *move the job to the data*, and the format permits it.

Everything is addressable. DATA, FUNCTION and JOB all have an identifier at the same offset, so a generic reader can identify, index and skip any of them without knowing which it is holding. REQUEST exists because that addressability is only useful if a node can ask.

There is deliberately no response type. A node that holds what was requested sends the object back as it stands — its header already states the identifier and the length, so nothing needs wrapping or restating — and a node that does not sends the same REQUEST back with its status field set. The asker distinguishes the two the way it distinguishes any two objects: by reading the type it was handed. There is no success status, because success is the object arriving.

8.3 Identity

Every identifier is a hash of the bytes it names — the first 64 bits of their SHA-256 digest — never a counter:

- `dataId` is the hash of the payload,
- `functionId` is the hash of the compiled object,
- `jobId` is the hash of the input identifier followed by the function identifier.

This is the same principle as Section 7.4, carried further. Four properties fall out of it, and the later architecture leans on all four.

Deduplication is free. Two heads that construct the same work arrive at the same job identifier, so a node can recognise that it has already done it.

Verification is free. A node that receives a FUNCTION can hash the bytes and compare them with the identifier it asked for. Nothing has to be trusted about the sender: the hash being cryptographic, a sender cannot choose other bytes that pass the check.

Lookup needs no directory. An artifact is stored under its own identifier as its file name — `functions/<functionId>.so`, `datasets/<dataId>.bin` — so resolving a reference is constructing a path, not searching. Both the head and the worker use this convention, which is also what makes a result usable the moment it is written.

Re-execution is safe. A job is a pure function over an immutable, content-addressed input. A job that does not come back can simply be run again somewhere else. There are no transactions, no locks and no two-phase commit anywhere in the design, and this is the reason.

The one place where content-addressing is not used is the predetermined `outputDataId`. The head assigns it when it creates a job, before the result exists, so that a consumer can name its input in advance. The result's own identifier is the hash of the bytes that actually came back. Both are defensible and the system currently keeps both; naming which is authoritative is an open question recorded in the implementation notes.

In the current implementation this is settled: a result is named by the job that produced it. `outputDataId` is the job's own identifier, the worker stores the result as `datasets/<jobId>.bin`, and that is also the key under which it is published to the cluster. The rule is what makes dependencies expressible without a table: a job that consumes another's result simply names that job's identifier as its input and carries the `DEPENDENT` flag. The identifier is known before the producer has run, so the consumer can be written into the same job file.

8.4 The node

A node is two processes on one machine: a C worker that executes, and a Java head that decides. They share the wire definition and nothing else. They communicate over a unix socket, and — importantly — in one direction only.

8.4.1 The worker

The worker is roughly three thousand lines of C. It listens, reads objects, and runs what it can.

- **The receiver** deserialises objects. DATA payloads are read *straight off the socket into the store's arena*, with no intermediate copy. FUNCTION code is written to disk, because `dlopen` wants a file. JOB objects are appended to a queue. REQUEST is currently parsed and not answered.

- **The object store** is a memory mapping with a bump allocator and two hash tables. The mapping is created with `MAP_NORESERVE`, so a four gigabyte capacity costs nothing until pages are touched. Records are carved from the same arena, which makes teardown one `unmap` rather than a walk.
- **The job queue** is a circular doubly linked list. Jobs are appended and never removed; what moves is the state, through `WAITING`, `READY`, `RUNNING` and `COMPLETED`. Keeping the job on the list is what allows a result arriving later to be matched to the job that asked for it.
- **The pool** is a work-stealing thread pool: one deque per thread, popped LIFO locally and stolen FIFO from siblings. Each thread owns its output buffer and shares nothing else.

A job becomes runnable when *both* halves are present: the `DATA` it names is in the store and the `.so` it names is on disk. The queue is swept after every object that arrives, because any arrival may be the missing half.

Functions are `dlopened` once and cached by identifier. In the two-node experiment the same 15.5KB object served three jobs on the receiving node and was loaded exactly once — the behaviour the earlier chapter listed as the shortest path to fewer bytes on the wire, now implemented on both sides.

In the current implementation the worker has been narrowed further.

- It serves *exactly one head*. It accepts one connection, removes its socket so that nothing else can connect, serves that head until it disconnects, and then drains the pool, reports any unfinished job and exits. `SIGINT` and `SIGTERM` are blocked in the pool threads so that they always interrupt the main thread's `accept` or `read`.
- Everything concerning code lives in one module, the function loader. `functionLoad(id)` opens `functions/<id>.so`, looks up `engineJob`, and caches the pointer by identifier under its own lock, so loading a large object no longer holds up the receiver. There are no symbol files.
- The division of a job into parts, and the collection and joining of their results, was removed. It divided payloads at byte boundaries, which none of the supplied functions could use, and nothing called it; the code is kept aside in `worker/parallelizable.bak`. A job always runs whole, and submitting a result is now a single step: store it, write the file, mark the job complete.
- Because the head now resolves every job before sending it (below), the worker in practice receives only jobs whose `.so` and data it already holds. The `WAITING` state remains for the order in which the objects of one job arrive, and lasts microseconds.

8.4.2 The filesystem as the reply path

The protocol has no reply. The worker never sends anything to the head. This is a deliberate simplification and it creates one problem: how does anything learn that a result exists?

The answer adopted here is that the worker writes each result into the same dataset directory the head reads as its catalogue, under the result's own identifier, via a temporary name and a rename so that a reader never sees a partial file. A file appearing *is* the notification. That single decision does three jobs at once: the result becomes usable as another job's input, it becomes servable to another machine, and it becomes publishable to the cluster — none of which required a message.

It is worth being honest that this is a convenience rather than a design principle. It works because both processes are on one machine and the filesystem is the cheapest shared medium available to them. A proper status channel from worker to head is the change that would make several other things correct at once, and Section 8.8 says which.

8.4.3 The head

The head is roughly three thousand lines of Java in a ports-and-adapters arrangement. Its domain classes are *views over bytes* rather than parsed copies: an object is a byte array and an offset, and every accessor reads where the contract says the field lives. Wrapping bytes that have just arrived is therefore a constructor call, not a decode, and an object read from one socket can be written to another with no copy in between.

The one piece of the format that must exist twice — once in C and once in Java — is *generated*. A small script reads `contract.h` and emits the Java constants, so the second copy is a build step rather than a habit. This directly addresses the drift risk identified in Section 7.8, where the two heads agreed only because someone kept them agreeing.

Three entry points exist, and the difference between them is only where jobs come from:

- one reads a job file or a FIFO,
- one listens on TCP for jobs pushed by another node,
- one takes a function and a payload on the command line.

They share a resolver, a catalogue and a submitter. A job arriving from any source is a JOB object by the time it reaches the resolver, so there is one resolution path and not three.

In the current implementation the head is organised strictly as a hexagon, and the dependency rule was checked mechanically: the domain imports nothing else in the head, the ports import only the domain, and the services only the domain and the ports.

- **Domain.** The four objects, the identifier types, `NodeLoad`, and `Wire` — the constants generated from `contract.h`, which moved from an adapter package into the domain because they are the contract.
- **Ports.** `ArtifactCatalog`, which now also keeps an in-memory index of the identifiers held, so that “is it here?” is a set lookup; `ArtifactRequester`, the way to ask the other head for something; `ClusterDirectory`; `LoadSampler`; and the object reader and writer.
- **Services.** `JobResolver` decides when a job can run, and nothing else. `WaitingJobs` is the list it walks: a singly linked list of unresolved jobs with a set of their identifiers beside it. `OpenRequests` records what has been asked of the other head and not yet received. `ArtifactExchange` stores the objects that arrive and answers the other head’s requests. `Presence` publishes load and holdings.
- **Adapters.** The files, the `etcd` client and `/proc`; `PeerNetwork` and `PeerLink` for the other head; and `HeadRuntime`, which only wires these together and runs a one-second tick.

The resolution rule is short enough to state whole. A job is *ready* when the head holds both its `.so` and its data; a ready job is sent to the local worker, or shipped. Otherwise it goes on the waiting list, and every sweep — once a second, and whenever an object arrives — walks the list, unlinks the jobs that have become ready, and for the rest asks the other head for whatever is missing. The one exception is a dependent job whose producer is this head’s own: its input will appear locally when the worker writes it, so the job waits without asking. A dependent job whose producer ran elsewhere asks for the result on every sweep until the other head has it.

The head reads its job file from a fixed place, `jobs/jobs.bin`, where a separate project, `pipeIn`, publishes it (Section 8.6).

8.5 The cluster layer

Two machines must answer two questions about each other: how loaded are you, and what do you hold. The system answers both through `etcd`, and answers nothing else through it.

8.5.1 What is stored

key	value
<code>/engine/nodes/<nodeId></code>	<code>cpu memory running slots address</code>
<code>/engine/data/<nodeId>/<dataId></code>	empty — the key is the fact

Jobs, payloads, function code and results never pass through `etcd`. They travel node to node over TCP. This distinction matters more than it may appear: `etcd` is a consensus store, so every write is a Raft round trip replicated to every member. Putting work items into it would make the coordination layer carry the system's throughput, which is precisely the load it is least suited to. The store carries what the *decision* needs, and the decision is small.

8.5.2 Leases, or how a node leaves

Each head grants a five second lease at startup and renews it once a second. Every key it writes is attached to that lease. A machine that stops reporting has its entries removed by `etcd` itself, within seconds, with no participant having to detect the failure or clean up after it.

The consequence is that presence in the store is a *live claim* rather than a record. A dump taken after the nodes have stopped is empty, and that is the intended behaviour. It also means the data map advertises only what is currently reachable: the bytes survive on a dead machine's disk, but advertising them would direct work at a node that cannot answer.

8.5.3 Publishing the difference

The holdings map was first implemented as one key per node containing the whole list of identifiers, rewritten every second. That is correct and it does not scale: nine identifiers is 152 bytes, but a node holding ten thousand results would push roughly 170 KB per second of almost entirely unchanged data through a consensus protocol.

It was therefore changed to one key per dataset, written when the dataset appears and deleted when it goes. The adapter keeps the set it has already published and writes only the difference; a node whose holdings have not changed writes *nothing*. Because the published set can drift from what the store actually holds, everything is rewritten once every thirty cycles, which repairs the drift without anyone having to detect it.

The measured effect in the two-node run: fifteen dataset keys were created by three publishing operations as the work completed, after which five further timer cycles wrote nothing at all.

8.5.4 Moving work

Placement is a head decision and remains so; a node never chooses its own work. Two policies are implemented.

The first ships a job when this machine is busy above a threshold *and* a peer is meaningfully lighter. The margin is not decoration: without it two machines at similar load pass jobs back and forth indefinitely, each becoming the lighter one in turn.

The second ignores load and hands jobs out in turn to every node with room. This is the right policy when machines are genuinely equivalent, and it is the only one that functions in a single-machine experiment, where both nodes read the same `/proc` and are therefore identical by construction. That limitation is discussed in Section 8.6 rather than hidden.

Load itself is sampled from `/proc/stat` as a delta between readings — genuine utilisation over the interval — and from `/proc/meminfo` as `MemAvailable` against `MemTotal`, fifteen samples at

one second each. An earlier implementation used the JVM heap and the one-minute load average; both were wrong, the first because it measured the wrong process and the second because a one-minute average cannot produce a fifteen-second window.

Fetching, in the current implementation. Moving work is no longer only pushing it. Each head holds a link to the other that is used in both directions. A `REQUEST` for a `.so` or a dataset is answered with the `FUNCTION` or `DATA` object as it stands, or with the same `REQUEST` marked `NOT_FOUND`; the asker then frees the identifier and asks again on a later sweep, which is how a dependent job waits for a result that the other node has not produced yet. Each identifier has at most one open request at a time. Requests never block the resolver: they are queued, and a dispatcher thread finds the other head — given on the command line, or the node `etcd` lists, never a client that connected to push jobs — and hands the request over. Every link, in turn, writes only from its own thread through a queue. This last point also removed a deadlock: when a link's reader wrote its own answers, two heads sending each other large objects at the same moment would both block writing and neither would read.

8.6 What the two-node experiment establishes

The experiment runs two complete nodes and one `etcd` instance on a single machine. Each node has its own socket, its own function directory and its own dataset directory; nothing is shared. Six jobs are read from a file on the first node.

Two claims are made and both are checked by evidence rather than assertion.

Node to store. Both nodes appear under `/engine/nodes/` with their own advertised addresses, and the first node can only have obtained the second's address by reading it there. After both stop, the store is empty, demonstrating the lease.

Node to node. The second node begins with two empty directories. It ends holding the shared object, its symbol file, the inputs it was sent and the results it produced. Since nothing is shared between them, every one of those files crossed a TCP socket. Its worker log shows the jobs arriving, resolving and completing.

Three jobs ran locally and three were shipped. The function was transferred once for all three, not three times.

What the experiment does *not* establish is the balancing policy. Two nodes on one machine read the same kernel counters and report the same load, so no peer is ever lighter and the load rule can never fire. The experiment uses the spreading policy for exactly this reason. Establishing the load rule requires two machines, and no claim is made about it here.

The current implementation, end to end

The system is now two projects. `engineJava` is the engine: the C worker and the Java head. `pipeIn`, beside it, produces work for it. `pipeIn` reads every wire offset from the engine's `contract.h` rather than restating it, compiles the job functions, generates their inputs, and publishes a job file to the place the head reads by default.

The functions. Two are supplied, `fib` and `factorial`. Each exports `engineJob` and works value by value on a dataset: a 24 byte header giving the number of rows and columns, then each row as its identifier followed by its values, where a row's identifier is the hash of its values. Fibonacci is defined up to 92 and factorial up to 20, the largest arguments whose results fit a signed 64-bit integer. Each compiles to a shared object that is named, like everything else, by the hash of its own bytes.

The jobs. Four jobs are generated from seeded random vectors: fibonacci over an 8×4 vector, factorial over another, fibonacci over a 4×4 vector of small values, and — the dependent case — factorial over *the result of* that last job. The dependent job carries no data; its input is the producer’s job identifier. The job file holds the FUNCTION, DATA and JOB objects back to back: nine objects, 32 896 bytes.

The run. On one machine, the head read the job file, sent the three independent jobs to the worker straight away, and held the dependent one. About a second later its sweep found the producer’s result in the dataset directory, where the worker had written it, and ran the dependent job. The worker loaded each function once, produced four results, and reported every job complete on shutdown.

The check. pipeIn computes every expected result in Python, independently of the engine, and compares it with the file the worker wrote. All four results were identical to the byte — including the row identifiers, which the C functions compute with the SHA-256 implementation in `contract.h`. That implementation was separately checked against a reference SHA-256 on 108 inputs, including every padding boundary.

What is not yet shown. This run used one node. The two-way link, the request and answer path and the NOT_FOUND retry are built, but have not yet been exercised between two nodes, and no claim is made about them until they have.

8.7 Technologies considered and not used

Several of the decisions above have well-established alternatives. Each is worth stating with the reason it was not taken, since a design is as much what it refuses as what it contains.

8.7.1 Raft, implemented rather than consumed

The system depends on distributed consensus and does not implement it. Raft [9] was designed explicitly for understandability relative to Paxos, and an implementation is a realistic undertaking: leader election, log replication, membership change and snapshotting.

It was not implemented, for three reasons.

First, the system needs consensus for a very small amount of state — node presence and holdings — and needs it to be *correct* rather than fast. A hand-written Raft would be the least tested component in the project while being the one whose failure is hardest to observe, since a split-brain does not crash anything, it merely produces two plausible answers.

Second, the properties actually relied upon are the ones an implementation gets wrong last: leases that expire without a coordinator, linearizable reads, and correct behaviour during a leader change. `etcd` provides these and is exercised continuously as the backing store of Kubernetes, discussed in Chapter 3.

Third, and most importantly, consensus is deliberately *off* the data path here. Nothing about a job, a payload or a result passes through it. Writing a Raft implementation to carry two keys per node per second would be a large investment in the part of the system that does the least work.

It is worth noting what an in-house implementation would have bought: control over the lease semantics, the ability to co-locate the store with the node and remove a process, and the removal of an external dependency from the deployment. For a thesis whose subject is the underlying mechanisms, that is not a negligible argument — it is simply outweighed by the risk of an unobservable correctness bug in the component everything else trusts.

8.7.2 ZooKeeper and Consul

Both solve the same problem. ZooKeeper’s ephemeral znodes are very close to the lease mechanism used here, and its watch primitive would remove the polling that the current design performs. It was not chosen mainly for operational weight: a JVM process with its own configuration and failure modes, in a project that already has a JVM component with different concerns. Consul adds service discovery and health checking that the system does not need, since a node’s presence claim already is its health check.

8.7.3 Gossip instead of consensus

A membership protocol such as SWIM [10] would remove the external store entirely. Nodes would exchange load and holdings directly, with failure detection by suspicion and dissemination by piggyback. It scales considerably better than a consensus store — no single quorum, no leader, load spread across participants — and for load and holdings, which are advisory and tolerate being seconds stale, eventual consistency is entirely sufficient.

This is the most defensible alternative and would be the first thing to try at a larger node count. It was not chosen because at two nodes it is strictly more code for strictly less certainty, and because the cluster state is currently read far more often than it is written.

8.7.4 A distributed hash table

For locating results, a DHT in the style of Chord [11] would replace the holdings map. Every node computes the same answer for “who should hold this identifier” from the identifier alone, and no directory is needed at all — a natural fit for a system that already content-addresses everything.

The system leans towards this without adopting it. What is implemented computes ownership as a hash modulo the node count, which is correct but reshuffles almost every key when a node joins. Rendezvous hashing would move only $1/N$ of the keys and is the obvious improvement. The full DHT with routing and replication is unnecessary at this scale, where any node can hold the addresses of all others.

8.7.5 A message broker

Kafka or RabbitMQ between the heads would provide durability, replay, consumer groups and back-pressure — the last of which the current design genuinely lacks. A job queue is, after all, a queue.

It was rejected because it inverts the property the system is built on. A broker is a store-and-forward intermediary: work is written to it and read from it, which means every job and, if payloads were included, every byte of data would traverse a third machine. The entire argument of Section 8.2 is that the job should travel to where the data already is. A broker in the middle makes that impossible by construction, and it is the same objection that rules out putting jobs into `etcd`.

8.7.6 gRPC, and other serialisation formats

Chapters 4 and 5 discuss gRPC and Protocol Buffers in detail, and a reasonable reader will ask why neither is used here. The answer is specific rather than dismissive.

Protocol Buffers are a *schema* format: fields are tagged, varint encoded, and the reader reconstructs a message into language objects. That is exactly right for evolving service interfaces and exactly wrong for this system’s hot path, where a payload is an opaque block of bytes that must reach a C function’s address space without being copied or reinterpreted. The format here

is a fixed binary layout with every offset a compile-time constant precisely so that a worker can read a payload straight off a socket into an arena and hand the function a pointer.

The price of that decision is real and should be stated: no schema evolution beyond a version field and reserved bytes, no generated code, no cross-language tooling, and a second hand-maintained copy of the format for the Java side — mitigated, but not eliminated, by generating it.

Cap'n Proto or FlatBuffers would have been the closer comparison, since both allow reading fields in place without a parse step, and both would have supplied schema evolution that the current format lacks. They were not used mainly because the format here is small enough to define completely in one header that the C side compiles against directly, which keeps a single authority over the layout.

8.7.7 MPI and the HPC tradition

For distributing computation across machines, MPI is the established answer, with decades of tuning and collective operations far better than anything here. It suits a different shape of problem: a fixed set of ranks, known at launch, running the same program in lockstep, typically on a dedicated interconnect. This system assumes nodes that come and go, jobs submitted at arbitrary times, and code shipped at runtime rather than linked in advance. The two models are not substitutes.

8.7.8 Existing distributed frameworks

Spark, Ray and Dask all implement a superset of what is built here — DAG scheduling, fault tolerance by lineage, data locality, and far better developer ergonomics. Building on one of them would have produced a more capable system sooner.

The thesis's subject, however, is the underlying mechanisms. Using Spark would have meant describing Spark rather than constructing a wire format, a placement rule and an execution model. The specific thing this work demonstrates — that identity derived from content removes coordination problems rather than solving them — is visible only when one builds the layer where that decision is made.

8.7.9 Kubernetes as the scheduler

Chapter 3 covers Kubernetes, and it is capable of placing work across machines. Its unit is a container, its scheduling latency is measured in seconds, and its model assumes long-lived services rather than short jobs. Using it to place a 96 byte job would be a category error. Its natural role here is the one it already has in the deployment: managing the lifecycle of the nodes themselves, inside which this engine schedules jobs at millisecond granularity.

8.7.10 A shared filesystem or object store

NFS, Ceph or S3 for the datasets would remove the transfer problem: every node sees every artifact. This trades a network transfer this system controls for one it does not, and it gives up the property that a node knows locally whether it can run a job. It also concentrates load precisely where the system tries to avoid it. For a cluster with an existing distributed filesystem the trade may well be worth it, and the catalogue port is the seam at which such a backend would attach.

8.7.11 Faster transports

RDMA would eliminate the copies in the transfer path; DPDK or AF_XDP — the latter discussed in Section 4.1.3 — would bypass the kernel network stack. All three are appropriate when the

transport itself is the bottleneck. On the hardware this system targets, an ordinary 1 Gb or 2.5 Gb link, the limit is the link and not the stack: 2.5 Gb is roughly 312 MB/s, and a single TCP stream saturates it using about one core with standard offloads enabled. The measures that matter at that scale are avoiding transfers altogether, batching small objects so that a 96 byte job does not occupy its own packet, and compressing payloads — LZ4 compresses at roughly an order of magnitude faster than the link can carry the result, so it is free headroom.

8.8 Limitations and what attaches next

In the spirit of Section 7.10, the omissions are stated plainly.

- **Nothing fetches a result.** A node can determine from the store which node holds a dataset, but cannot ask for it. Until it can, a job whose input is another job's output can only run where that output already is. This is the largest remaining gap: it is what would turn two machines running separate work into one system.
- **The worker never reports.** There is no path from worker back to head. Consequently the running-job count published to the cluster is always zero, a failed job is invisible, and a job waiting on a *result* is not woken until some unrelated object arrives. The filesystem covers results by side effect rather than by design.
- **A job has exactly one input.** A job fed by two producers has no representation. Fan-in exists only for the parts of a single divided job. The natural extension is to let several producers write parts of one named input, reusing the parent and part-index machinery that already rebuilds splits.
- **Shipping duplicates payloads.** Sending a job together with its function and data means the payload then exists on both machines. Sending only the 96 byte job requires the receiver to be able to ask for what it lacks, which is the same missing capability as the first item.
- **No back-pressure.** The pending queue is unbounded and the pool drops a task when every deque is full. One fast node can bury a slow one. The refusal status already exists in the protocol as the shape of the answer.
- **Division is unsound for the current functions.** The splitting machinery cuts a payload at byte boundaries, which is meaningless to a function that reads a header first — which all of the supplied functions do. Either the head understands the payload format, reintroducing the coupling that opaque payloads removed, or the shared object supplies its own splitter and merger alongside its job function. The second is consistent with the rest of the design: the code that understands the bytes is already shipped with them.
- **Security is absent by construction.** The worker `dlopen`s bytes that arrived over a socket. That is remote code execution as a feature, with no authentication anywhere in the protocol. On a unix socket the filesystem permissions are the entire security model. The moment the worker accepts objects over TCP, that ceases to be true, and an authenticated handshake becomes a prerequisite rather than an enhancement.

Since these were written. Of the items above, two have changed. *Fetching* now exists in the head: a node asks the other head for a `.so` or a dataset and is answered with the object or with `NOT_FOUND`, and a dependent job asks for a result produced elsewhere until it exists. It has been built and not yet measured between two machines. *Division* was removed rather than repaired; a job always runs whole. The worker still never reports, but a job waiting on a result no longer depends on an unrelated arrival to wake: the head's one-second sweep finds the result on disk.

Shipping still sends the data with the job; sending only the 96 byte job now works in principle, since the receiver can ask for what it lacks. Back-pressure and security are unchanged.

What the system does establish is narrower than a production scheduler and, I would argue, more durable: a wire format in which framing, identity and addressing are properties of the bytes rather than of the protocol around them; an execution model in which work is idempotent by construction, so that recovery is recomputation and needs no transactions; a coordination layer that carries decisions rather than data, and in which membership expires by itself; and a demonstrated transfer of work between two independent machines that shared nothing but the definition of the format.

Each of the omissions above attaches to that structure without disturbing it, which — as in the previous chapter — was the objective.

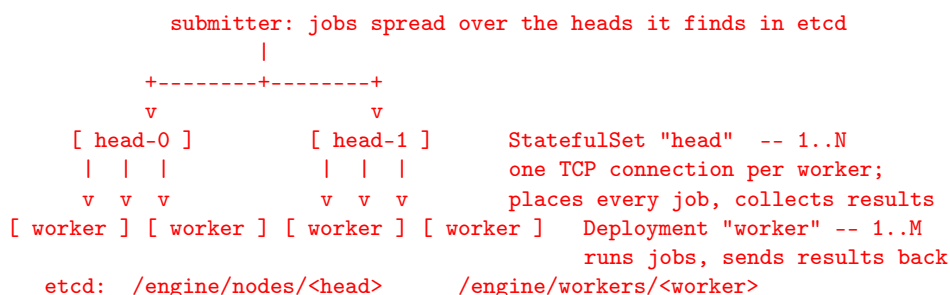
Chapter 9

Scaling the Parts Apart, and a Controller of Our Own

The deployments so far keep the node of Section 8.4 whole: one pod is one head and one worker, and the engine grows by adding whole nodes. This chapter takes the two steps that the rest of the thesis points towards. The first splits the node, so that heads and workers become separate pods that Kubernetes scales independently. The second asks whether Kubernetes is the right tool for managing the engine's processes at all, and makes the case — with its costs — for a controller of the engine's own, built from the systems programming of Chapter 2 rather than from an orchestrator. It closes with the two tools that join that controller to the cluster around it: KEDA, to scale on the engine's own load, and CRIU, to move a process rather than lose it. Last, it names three notions from distributed systems that the engine would benefit from implementing.

9.1 Heads and workers as separate pods

Section 3.5.9 sketched this split as a design. It has since been built, as `engineHeadWorkers`, and generalised from one head to several: a pod is *either* a head *or* a worker, and each kind is its own unit of scale.



9.1.1 Why the two parts want to scale differently

The two halves of a node are loaded by different things. A worker is bound by CPU: its cost grows with the work inside each job. A head is bound by coordination: resolving jobs, holding a catalogue, answering requests and placing work — its cost grows with the *number* of jobs and of connections, and hardly at all with how long each job runs. Scaling them together, as whole nodes, adds a head for every worker the CPU demands, and a worker for every head the job rate demands. Splitting them lets each follow its own load.

9.1.2 What the engine needed

The node relied on two things that hold only inside one pod, and both had to change.

The worker gained a listener and a reply. It still accepts the unix socket, unchanged, and now also listens on TCP, serving each connection on a thread of its own, since several heads hold one each. A worker that shares no directory with its head cannot answer by writing a file, so every result goes back on the connection its job arrived on, as an ordinary DATA object flagged `DATA_FLAG_RESULT`. That is the protocol's first reply, and it needed no new framing: the objects of Section 8.2 are valid on any byte stream. A connection is reference-counted — the reader holds one reference and every job that arrived on it another, until its result has been sent — and writes from the pool's threads are serialised.

The worker registers itself. Once a second it renews a five-second lease and writes `/engine/workers/<id>` — its address, threads, and active and completed counts — through etcd's JSON gateway, with no client library. On `SIGTERM` it *revokes the lease first*, so that heads stop sending to it at once, then finishes what it holds and sends the results back before it exits.

The head became a dispatcher. It keeps one long-lived connection per worker it finds in etcd, re-read every second, and places each job itself: on the worker with the most free threads, never more of its jobs on a worker than that worker has threads, so nothing queues on a busy worker while another is idle. Results that come back are stored in the head's catalogue and published as before. A worker that leaves etcd is sent nothing new but its results are still awaited; a worker whose connection *closes* has every job it still held put back at the front of the queue for another worker.

9.1.3 How it is deployed

object	role
StatefulSet head	the heads; a stable name each, to advertise; a PersistentVolumeClaim each for its catalogue, so results outlive restarts
headless Service head	one DNS name per head
Deployment worker	the workers; interchangeable, since only heads talk to them and each advertises its pod address in etcd; 90s to finish their jobs when stopped
ScaledObject worker	KEDA on the workers' CPU, 1 to 8 pods; up to four new workers every 30s, then down one a minute after two quiet minutes
ScaledObject head	KEDA on the heads' CPU, 1 to 3 pods, scaled slowly because a head holds results

A Deployment suits the workers precisely because the split removed what made a node need a StatefulSet: no one dials a worker by a name it advertised for others, only the heads, which learn its address from etcd.

9.1.4 What it changes, and what was shown

	one pod per node	heads and workers apart
a pod added by KEDA	gets work from the <i>next</i> submission	gets work within a second: heads place per job
a pod scaled down	its queued jobs and results are lost	a worker finishes, or its jobs are re-sent; results are on the heads' volumes
a worker crashing	silent	seen: its connection closes, its jobs go elsewhere
where results are	spread over every pod	on the heads

Run as processes on one machine, with etcd, the split behaved as designed: twelve jobs over two heads and three workers were placed six per head and three, four and five per worker, every result byte-identical to calling the function directly; a worker killed outright under eighteen jobs had its four unfinished jobs re-sent by both heads to the other workers, and all eighteen results arrived; a worker added mid-batch was connected to and given jobs by both heads within half a second; and a worker sent `SIGTERM` mid-batch left etcd, finished its jobs and returned every result. The manifests and scripts were checked, but the Kubernetes deployment itself had not been run when this was written.

9.1.5 What it costs

Every job's data now crosses the network from a head to a worker, where a whole node runs work on data it already holds, over a local socket; a head's link becomes a throughput limit, and on two machines half the workers sit across the network from any given head. Results live on the heads, so a head scaled down takes its results offline until it returns. And each head counts only its own jobs against a worker's threads: with several heads a worker can be sent more than it runs at once. The workers publish their active count, which a head could subtract, but does not yet.

The split is also where Kubernetes' limits start to show. Both units scale on CPU, a proxy for load that the engine could measure directly — the number of jobs queued and running — but that the autoscaler cannot see. KEDA polls every fifteen seconds and the HPA smooths its decisions over windows of tens of seconds, while a new job is placed in a millisecond. The next section starts from that mismatch.

9.2 The case for a controller of our own

Kubernetes manages *containers*: it starts them, restarts them, gives them names and addresses, and changes how many there are, all by reconciling a declared state against the observed one through its API server and etcd (Chapter 3). The engine, by contrast, cares about *processes and jobs*: which worker holds which job, how many are queued, which process died holding what. The two views meet only at the edges, and every decision that crosses from one to the other pays for the gap.

9.2.1 What the orchestrator costs this engine

- **It cannot see the load.** Autoscaling reads CPU through metrics-server, because that is what it can observe. The quantity that matters — jobs waiting for a thread — lives inside the engine, and has to be approximated by the CPU it eventually causes. Chapter 3 showed the consequence: with one thread per pod, a busy pod reads at most 100 %, so a 70 % target could only creep the cluster from two pods to three.
- **It is slow by design.** A control loop that polls every fifteen seconds, smooths over minutes, pulls an image and starts a container is right for services that change size a few times a day. Jobs arrive and finish in milliseconds.
- **It forces the network between parts.** Two containers in one pod share volumes and loopback; two pods share nothing but the network. The engine's natural channels on one machine — a unix socket, a shared file, a shared mapping — stop at the pod boundary, and the split of Section 9.1 replaced all of them with TCP.
- **A restart is blind.** When a container dies, Kubernetes restarts it; it does not know that the process held six jobs, and cannot give them to anyone else. The engine had to rediscover that itself, from a closed connection.

None of these is a defect of Kubernetes. They are the price of its generality, paid here by an application whose units of work are far smaller and faster than the units Kubernetes manages.

9.2.2 The same responsibilities, from the operating system

Everything an orchestrator does for one machine can be done by a single process that starts the others and watches them, using the primitives of Chapter 2. Two earlier versions of the engine did exactly that.

engineJavaCore: a supervising controller. Its controller creates one POSIX shared-memory arena (`shm_open`, `ftruncate`, `mmap` with `MAP_SHARED`) holding everything the node shares: a ring-buffer job queue per worker and a backlog, the job table, the data index and heap, a member table describing every process, and a mailbox. It starts every member with `fork` and `exec`, passing its role, the arena's name and an already-listening socket in the environment, and sets `PR_SET_PDEATHSIG` so that a member is signalled if the controller itself dies. It waits on a `signalfd` for `SIGCHLD` and `SIGUSR1`:

- on `SIGCHLD` it reaps with `waitpid`, moves the dead worker's queue and in-flight jobs to the backlog — the restart is *not* blind, because the jobs a process held are in shared memory — and restarts it in the same slot, unless it died within five seconds of starting;
- on `SIGUSR1` it answers the command a client, `enginectl`, has written into the mailbox (`status`, `add-pair`, `add-worker`, `remove`, `autoscale`, `stop`), replying through the same memory;
- with autoscaling on, it adds a worker whenever more jobs are queued than there are threads to run them, at most one every two seconds, and retires the newest autoscaled worker after fifteen seconds with nothing queued or running.

The arena's lock is a pthread mutex made `PTHREAD_PROCESS_SHARED` and `PTHREAD_MUTEX_ROBUST`: a process that dies holding it does not wedge the others — the next locker receives `EOWNERDEAD`, marks the mutex consistent and carries on. Workers take jobs from their own queue, then the backlog, then steal from the deepest other queue, and read their input in place from the shared heap.

engineLinked: a controller that links. Here the worker is broken into components — ingress, scheduler, executors, collector — that declare in a manifest which endpoints they export and which they import, like an object file's defined and undefined symbols. The controller resolves every import against an export before starting anything, and an unresolved one is a link error; it then creates the shared segments and a link table holding a pair of single-producer, single-consumer rings per link and a doorbell — a process-shared semaphore — per process, and `forks` and `execs` each component with its link numbers in its environment. No sockets remain between processes on one machine: a message is one copy into pages the other process maps, a busy link makes no system call at all, and a component waits on all its links by sleeping on one semaphore. When a component dies, the controller closes every ring it was an end of and rings the doorbells on the other side — what the kernel does for a socket — so its peers read the end of the stream.

9.2.3 Kubernetes, and what replaces it on one machine

Kubernetes	a controller of our own
desired state in the API server and etcd	the member table in shared memory
kubectl to the API server	enginectl: a mailbox in the arena, then SIGUSR1
kubelet starting a container	fork + exec, configuration in the environment
a container restarted on exit	SIGCHLD through signalfd, waitpid, restart in the same slot — and its jobs requeued
liveness probes	the process table itself; a robust mutex's EOWNERDEAD
a Service and its DNS name	an inherited listening socket; a named shared-memory endpoint
the HPA and KEDA, on CPU	scaling on the queue depth the controller reads directly
the pod network	shared rings and a shared heap: no copy through the kernel
the controller dying	PR_SET_PDEATHSIG: every member is signalled

9.2.4 The case, and its limits

The argument is not that a custom controller should replace Kubernetes. It is that each is right at a different level. Kubernetes is good at machines, images, rollouts and restarting whatever stops, across a cluster; it has no equivalent on one machine for the engine's controller, which starts, watches, requeues and scales the engine's own processes at the granularity of jobs, with the load it can read directly and the channels the operating system offers inside one machine. The two compose: Kubernetes places one pod per machine, as the current deployment already does, and inside that pod the controller runs the node.

The costs are real, and they are the costs of leaving a general tool.

- **It is one machine.** Shared memory does not cross machines; between them the engine still needs TCP and etcd, and Kubernetes still places the pods.
- **Isolation is weaker.** Processes sharing an arena share its fate: a worker that scribbles over the mapping can corrupt the others' state. A container boundary prevents that; here, cgroups and separate mappings would have to be added by hand.
- **It is Linux.** signalfd, PR_SET_PDEATHSIG and robust process-shared mutexes are not portable.
- **Supervision is ours to get right.** Restart storms, orphaned shared memory and signals that merge are problems an orchestrator has solved; a custom controller has to solve them again, and test them.

Against these, the engine gains a control loop that reacts in the time it takes to read a counter rather than a polling interval, scaling on the load that actually matters, failures whose consequences — the jobs a process held — are repaired rather than merely restarted, and data paths without a copy through the kernel. For a system whose unit of work is a millisecond-long job rather than a long-running service, that is the level at which control belongs.

9.3 Bridging the two levels: KEDA and CRIU

A controller inside each machine and Kubernetes across machines leave two gaps between them: the cluster still decides how many machines' worth of nodes to run without seeing the engine's load, and a process that must move between machines still loses whatever it was doing. Two existing tools close them, each at its own level.

KEDA: scaling the cluster on the engine’s own number. KEDA was used throughout Chapter 3 with its CPU trigger, because CPU was what the cluster could observe. KEDA does not require that: besides CPU it can read a value from an HTTP endpoint (its *metrics API* scaler), from a Prometheus query, or from an external scaler that it calls over gRPC. The controller of Section 9.2 already holds the quantity that matters — how many jobs are queued, and how many threads are running them — in its member table. Publishing that one number, for example on a small HTTP endpoint beside `enginectl`, lets KEDA scale pods on the engine’s real load instead of on the CPU it eventually causes. The two levels then divide the work cleanly: inside a pod, the controller adds and retires worker processes within seconds, at the granularity of threads; across the cluster, KEDA adds and removes whole pods — whole machines’ worth of capacity — more slowly, when the controllers’ queues say the machines themselves are full.

CRIU: moving a process instead of losing it. Kubernetes restarts what stops, and the controller requeues the jobs a dead worker held; either way, a job interrupted mid-computation starts again from nothing. CRIU (Checkpoint/ Restore In Userspace) removes that loss: it freezes a process tree, writes its complete state to disk — memory, registers, file descriptors, and with its TCP repair mode even established connections — and restores it later, on the same machine or another, continuing from the instruction where it stopped. Kubernetes exposes this through the kubelet’s checkpoint API (the forensic container checkpointing feature), which uses CRIU underneath; restoring such a checkpoint is not yet a first-class Kubernetes operation, and is done by building an image from the archive and starting it as a new container.

The earlier `ckpt` tool, a checkpoint/restore skeleton written to make a process’s state concrete, shows why CRIU and not a smaller tool is needed here: it supports one single-threaded process with no sockets, no shared memory, no `MAP_SHARED` mapping and no IPC of any kind — which excludes precisely the mechanisms the controller’s design is built on. A node of the IPC design is a *process tree* around a shared arena, so it is checkpointed as a tree, from the controller down, with its shared-memory segments; CRIU handles both, and a single worker cannot be checkpointed apart from the arena it shares.

That also sets the limits of where CRIU is worth its cost. For the millisecond jobs of this engine, draining is cheaper than checkpointing: a worker told to stop finishes what it holds, as the split deployment’s workers already do, and nothing is lost. CRIU earns its place for the long job — the one that has computed for an hour when its machine must be emptied — and for moving a whole node between machines as one unit, arena and all, rather than rebuilding it from the cluster’s state. Consistent with the earlier caution about relocating healthy containers, it is a tool for the exceptional move, not for routine scaling.

9.4 Notions from distributed systems worth implementing

The engine already rests on several ideas from distributed systems — content-derived identity, leases, a consensus store for membership (Chapter 6). Three more would repair specific weaknesses this thesis has named, each at modest cost.

Fencing tokens, for work that must not run twice. A head that loses its connection to a worker re-sends the worker’s jobs elsewhere; but a connection can fail while the worker is still alive and still running them, and the same job then completes twice. Because jobs are pure functions over immutable inputs, the duplicate is harmless to the *result* — the same bytes under the same identifier — but it is not harmless to anything with a side effect, and it wastes a worker. The standard remedy pairs the lease of Section 6.3 with a *fencing token*: each time a job is assigned, the assignment carries a number that only increases — the revision `etcd` returns when the assignment is written is exactly such a number — and whoever accepts the job’s result, or any effect it has, refuses a token older than the newest it has seen. At-least-once delivery then

produces exactly-once effects, which is the strongest guarantee a system without distributed transactions can offer.

Rendezvous hashing, for placing work near its data. Placement today is either the sender’s round robin or the head’s choice of the least-loaded worker; neither looks at where a job’s input already is, so a dependent job often runs on one machine while its input sits on the other, and has to be fetched. Rendezvous hashing (Section 6.7) gives every node the same answer to “which node owns this identifier” — the node with the highest $h(\text{node}, id)$ — without a directory and with minimal movement when nodes join or leave. Applied to results, and with a job placed by its *input’s* identifier, a dependent job would be sent to the node that produced its input: the rule the object model was designed to permit, *move the job to the data* (Section 8.2), made the default.

Better failure detection, and results that survive a node. A lease answers “is this node alive” with a fixed timeout, which is either slow to notice a crash or quick to mistake a pause for one. An accrual failure detector instead reports a continuous suspicion computed from the observed distribution of heartbeat intervals, so that each consumer — a head deciding whether to re-send, the controller deciding whether to restart — can choose its own threshold; and SWIM’s indirect probing (Section 6.5) asks other members to check a silent node before declaring it dead, so that one bad link does not look like a failure. The second half of the problem is that a result lives on exactly one node, and goes offline with it. Writing each result to more than one node, and reading from any node that has it — with $W + R > N$ when copies must be consistent, or simply “any copy” since a result’s content is fixed by its identifier — would let a head be scaled down, or a machine be lost, without the results it held disappearing with it.

Bibliography

- [1] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.
- [2] T. Kaimasidis, *Advanced C: the mechanics layer*, and the companion documents on concurrency, processes and system calls, unpublished notes, 2026.
- [3] B. Burns, J. Beda and K. Hightower, *Kubernetes: Up and Running*, O'Reilly Media.
- [4] B. Burns, *Designing Distributed Systems*, O'Reilly Media, 2018.
- [5] N. Poulton, *The Kubernetes Book*, self-published.
- [6] B. Gregg, *BPF Performance Tools*, Addison-Wesley, 2019.
- [7] L. Rice, *Learning eBPF*, O'Reilly Media, 2023.
- [8] Google, *Protocol Buffers Documentation*. <https://protobuf.dev/>
- [9] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm”, *USENIX Annual Technical Conference*, 2014.
- [10] A. Das, I. Gupta and A. Motivala, “SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol”, *IEEE DSN*, 2002.
- [11] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek and H. Balakrishnan, “Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications”, *ACM SIGCOMM*, 2001.
- [12] M. J. Fischer, N. A. Lynch and M. S. Paterson, “Impossibility of Distributed Consensus with One Faulty Process”, *Journal of the ACM*, 32(2), 1985.
- [13] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-tolerant Web Services”, *ACM SIGACT News*, 33(2), 2002.
- [14] L. Lamport, “The Part-Time Parliament”, *ACM Transactions on Computer Systems*, 16(2), 1998.
- [15] P. Hunt, M. Konar, F. P. Junqueira and B. Reed, “ZooKeeper: Wait-free Coordination for Internet-scale Systems”, *USENIX ATC*, 2010.
- [16] J. Kreps, N. Narkhede and J. Rao, “Kafka: a Distributed Messaging System for Log Processing”, *NetDB*, 2011.
- [17] D. Karger, E. Lehman, T. Leighton, R. Panigrahy, M. Levine and D. Lewin, “Consistent Hashing and Random Trees”, *ACM STOC*, 1997.